

Lab 2: Signal Analysis, Sampling, Aliasing, and Filtering

Names: Michael Harb, Spencer Neill, Marc Pham

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

rng = np.random.default_rng(7377)
```

3. Task 1: Statistical Properties of Signals

```
In [25]: A = 0.5
f0 = 10
fs = 32 * f0
Tobs = 4
N = int(fs*Tobs)
n = np.arange(N)
t = n / fs

x = A * np.cos(2 * np.pi * f0 * t)
```

3.1: Mean, variance, and power

```
In [3]: # Change this value to graph a different amount of time
plot_seconds = 0.5

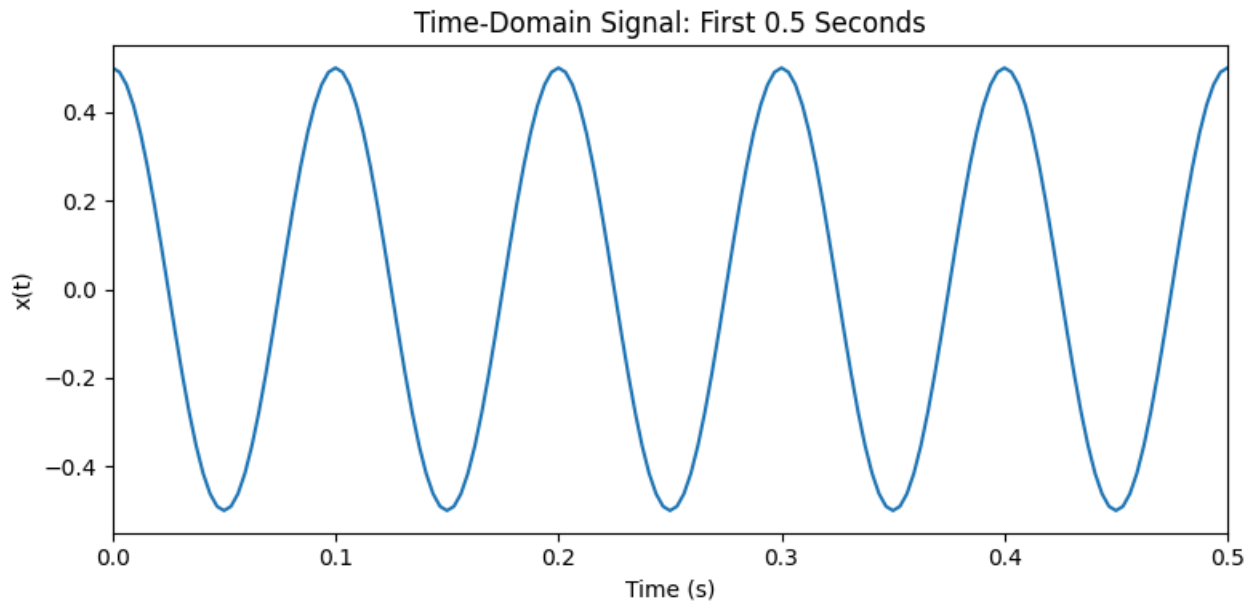
# Select the portion of x(t) to plot
mask = t <= plot_seconds

fig, ax = plt.subplots(figsize=(8, 4))

ax.plot(t[mask], x[mask])

ax.set_xlim(0, plot_seconds)
ax.set_xlabel("Time (s)")
ax.set_ylabel("x(t)")
ax.set_title(f"Time-Domain Signal: First {plot_seconds} Seconds")

plt.tight_layout()
plt.show()
```



The real record $x[n] = A \cos(2\pi f_0 t)$ is a sinusoid with 5 complete cycles over the first 0.5 seconds. Each record starts and ends at equivalent points in the sinusoid, the positive and negative portions cancel exactly for the mean.

The number of samples $N = f_s T = 32 \text{ samples} * 10 \text{ Hz} * 0.5 \text{ seconds} = 160$ samples for the full 4 second record $x[n]$.

Thus, over the first 0.5 seconds, the finite-record mean $\hat{\mu}_x = \frac{1}{N} \sum_{n=0}^{N-1} A \cos(2\pi f_0 t) = 0 \text{ V}$.

The finite variance and finite power are $\sigma_x^2 = P_x = \frac{A^2}{2} = \frac{0.5^2}{2} = 0.125 \text{ V}^2$.

The deterministic calculations are run below. Rounding was added to the 12th decimal places to account for Python-based round-off error.

```
In [4]: # Finite-record statistics for first 0.5 seconds
x_05 = x[t < 0.5]

mean_x = np.mean(x_05)
variance_x = np.mean((x_05 - mean_x)**2)
power_x = np.mean(x_05**2)

print("N =", len(x_05))
print("Finite-record mean =", round(mean_x, 12), "V")
print("Finite-record variance =", round(variance_x, 12), "V^2")
print("Finite-record power =", round(power_x, 12), "V^2")
```

```
N = 160
Finite-record mean = 0.0 V
Finite-record variance = 0.125 V^2
Finite-record power = 0.125 V^2
```

The number of samples $N = f_s T = 32 \text{ samples} * 10 \text{ Hz} * 4.0 \text{ seconds} = 1280$ samples for the full 4 second record $x[n]$.

The full record $x[n]$ goes for a full 4 seconds and is a sinusoid with 40 complete cycles. Rerunning the deterministic calculation below does not change the mean, variance, and power results because both the 0.5 second and 4 second records contain an integer number of complete cycles of the 10 Hz sinusoid. Thus, both records are zero-mean sinusoids with an integer number of complete cycles.

The theoretical mean $\hat{\mu}_x = \frac{1}{N} \sum_{n=0}^{N-1} A \cos(2\pi f_0 t) = 0 \text{ V}$.

The theoretical variance and theoretical power are $\sigma_x^2 = P_x = \frac{A^2}{2} = \frac{0.5^2}{2} = 0.125 \text{ V}^2$.

```
In [5]: # Theoretical statistics
mean_x2 = np.mean(x)
variance_x2 = np.mean((x - mean_x2)**2)
power_x2 = np.mean(x**2)

print("N =", len(x))
print("Finite-record mean =", round(mean_x2, 12), "V")
print("Finite-record variance =", round(variance_x2, 12), "V^2")
print("Finite-record power =", round(power_x2, 12), "V^2")
```

```
N = 1280
Finite-record mean = -0.0 V
Finite-record variance = 0.125 V^2
Finite-record power = 0.125 V^2
```

3.2: FFT

```
In [26]: X = np.fft.rfft(x)
A = np.abs(X) / N

# Double all positive-frequency bins except DC and Nyquist
if N % 2 == 0: A[1:-1] *= 2 # even N, Nyquist bin exists
else: A[1:] *= 2 # odd N, no Nyquist bin

f = np.fft.rfftfreq(N, d=1/fs)
```

The frequency-bin spacing $\Delta f = \frac{f_s}{N} = 320 \text{ samples per second} / 1280 \text{ samples} = 0.25 \text{ Hz}$.

```
In [27]: delta_f = fs / N
print(f"FFT-bin spacing: {delta_f:.3f} Hz")
```

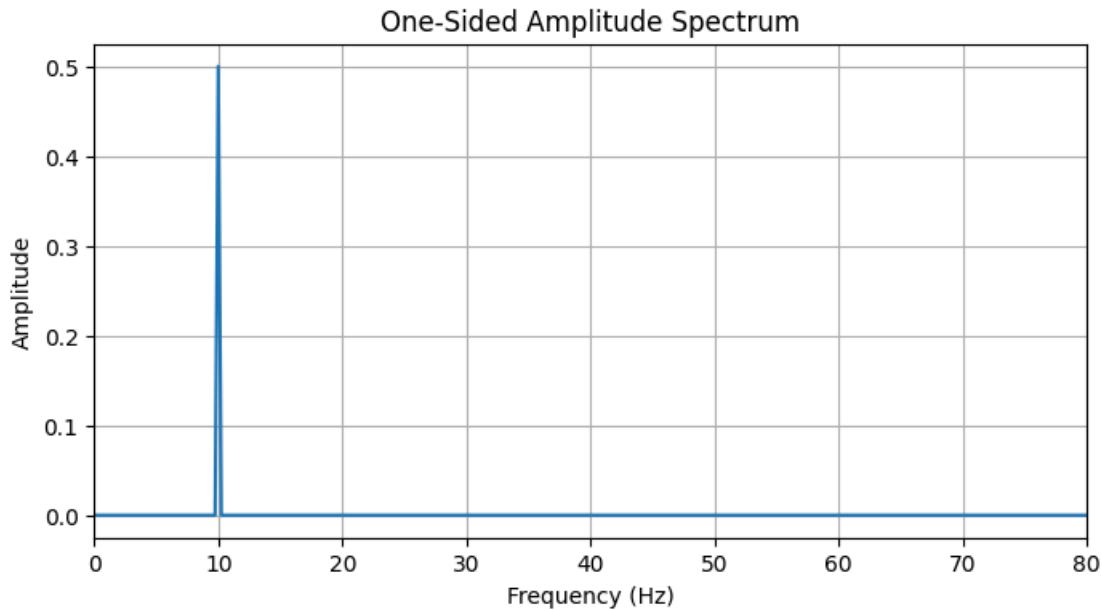
FFT-bin spacing: 0.250 Hz

Plotting the one-sided amplitude spectrum reveals a 10 Hz component with a measured amplitude of 0.5, which is double the calculated frequency-bin spacing because the positive-frequency bins were doubled.

```
In [28]: idx = round(f0 * N / fs) # Frequencies increase by an interval of fs / N from 0 to fs / 2
print(f"Measured {f0:g} Hz amplitude: {A[idx]:.6f} V")
```

Measured 10 Hz amplitude: 0.500000 V

```
In [11]: # Plot
plt.figure(figsize=(8, 4))
plt.plot(f, A)
plt.xlabel('Frequency (Hz)')
plt.ylabel('Amplitude')
plt.title('One-Sided Amplitude Spectrum')
plt.grid(True)
plt.xlim(0, 80) # adjust as needed
plt.show()
```



The deterministic FFT does not change when rerun because the input signal is exactly the same each time. The sinusoid $x(t) = 0.5\cos(2\pi 10t)$ is generated from fixed parameters (amplitude, frequency, sampling rate, and duration) with no random component or noise. Since the FFT is a deterministic mathematical operation, applying it to the same signal always produces the same frequency spectrum. Therefore, the location and amplitude of the 10 Hz peak remain identical every time the calculation is repeated.

3.3: Noisy Channel

The theoretical power of the signal $x[n]$ is $P_x = 0.125 \text{ V}^2$, as derived in Section 3.1

With a target SNR of 0 dB, we can calculate the required noise power with...

$$P_n = \frac{P_x}{10^{\text{SNR}_{\text{dB}}/10}} = \frac{0.125\text{V}^2}{10^{0/10}} = 0.125\text{V}^2$$

For a zero-mean AWGN, the noise power is equal to the noise variance, so $\sigma_n^2 = P_n = 0.125 \text{ V}^2$. Thus, for a target SNR of 0 dB, the required noise variance of the noise signal should be 0.125 V^2 .

The measured noise variance was 0.123528 V^2 , which is within about 1.18% of the required noise variance. The measured SNR was 0.0509 dB, very close to the desired 0 dB. Small differences occur because the generated AWGN is finite set of random noise samples, so its sample statistics do not exactly match the theoretical values.

```
In [46]: # Signal power of the clean sinusoid
signal_power_v2 = np.mean(x**2)

# 0 dB target SNR
noise_power_v2 = signal_power_v2

# Standard deviation corresponding to the required variance
noise_standard_deviation_v = np.sqrt(noise_power_v2)

rng = np.random.default_rng(7377)

# Generate AWGN realization
noise_samples = noise_standard_deviation_v * rng.standard_normal(N)

# Received signal
noisy_signal = x + noise_samples

# Measured noise variance
```

```

measured_noise_variance_v2 = np.var(noise_samples)

# Measured SNR
measured_snr_db = 10 * np.log10(
    np.mean(x**2) / np.mean(noise_samples**2)
)

print(f"Required noise variance: {noise_power_v2:.6f} V^2")
print(f"Measured noise variance: {measured_noise_variance_v2:.6f} V^2")
print(f"Measured SNR: {measured_snr_db:.4f} dB")

```

Required noise variance: 0.125000 V²
 Measured noise variance: 0.123528 V²
 Measured SNR: 0.0509 dB

```

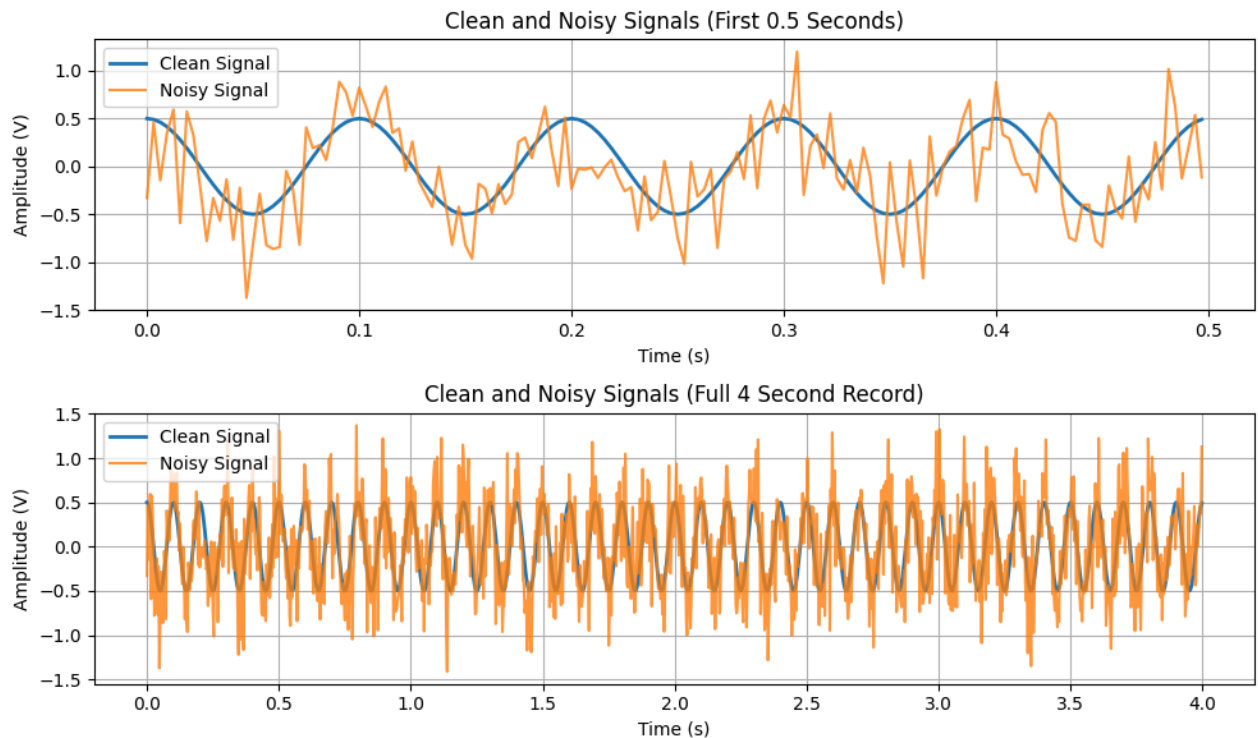
In [47]: plt.figure(figsize=(10, 6))

# First 0.5 s
plt.subplot(2, 1, 1)
plt.plot(t[:160], x[:160], label="Clean Signal", linewidth=2)
plt.plot(t[:160], noisy_signal[:160], label="Noisy Signal", alpha=0.8)
plt.title("Clean and Noisy Signals (First 0.5 Seconds)")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude (V)")
plt.grid(True)
plt.legend()

# Full 4 s record
plt.subplot(2, 1, 2)
plt.plot(t, x, label="Clean Signal", linewidth=2)
plt.plot(t, noisy_signal, label="Noisy Signal", alpha=0.8)
plt.title("Clean and Noisy Signals (Full 4 Second Record)")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude (V)")
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()

```



```

In [61]: # Voltage axis for theoretical PDF
v = np.linspace(-1.5, 1.5, 1000)

```

```

# Target Gaussian PDF
gaussian_pdf = (
    1 / np.sqrt(2 * np.pi * noise_power_v2)
) * np.exp(
    -(v**2) / (2 * noise_power_v2)
)

plt.figure(figsize=(13, 5))

# Histogram of finite noise realization
plt.hist(
    noise_samples,
    bins=36,
    density=True,
    alpha=0.7,
    label="Finite Noise Realization"
)

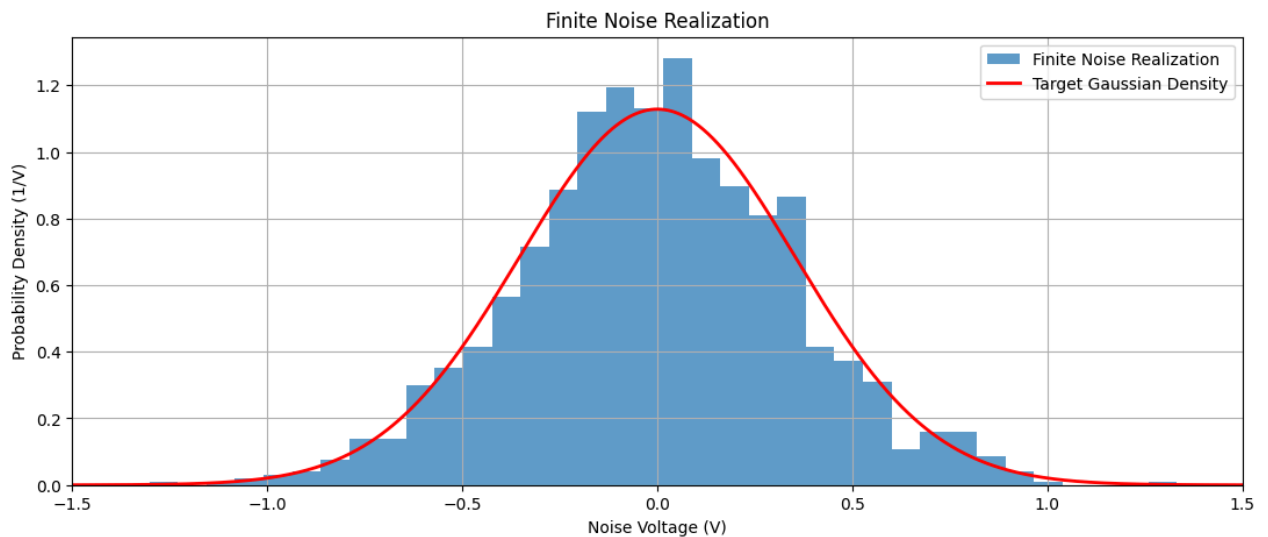
# Theoretical Gaussian PDF
plt.plot(
    v,
    gaussian_pdf,
    'r',
    linewidth=2,
    label="Target Gaussian Density"
)

plt.xlim(-1.5, 1.5)

plt.xlabel("Noise Voltage (V)")
plt.ylabel("Probability Density (1/V)")
plt.title("Finite Noise Realization")
plt.grid(True)
plt.legend()

plt.show()

```



```

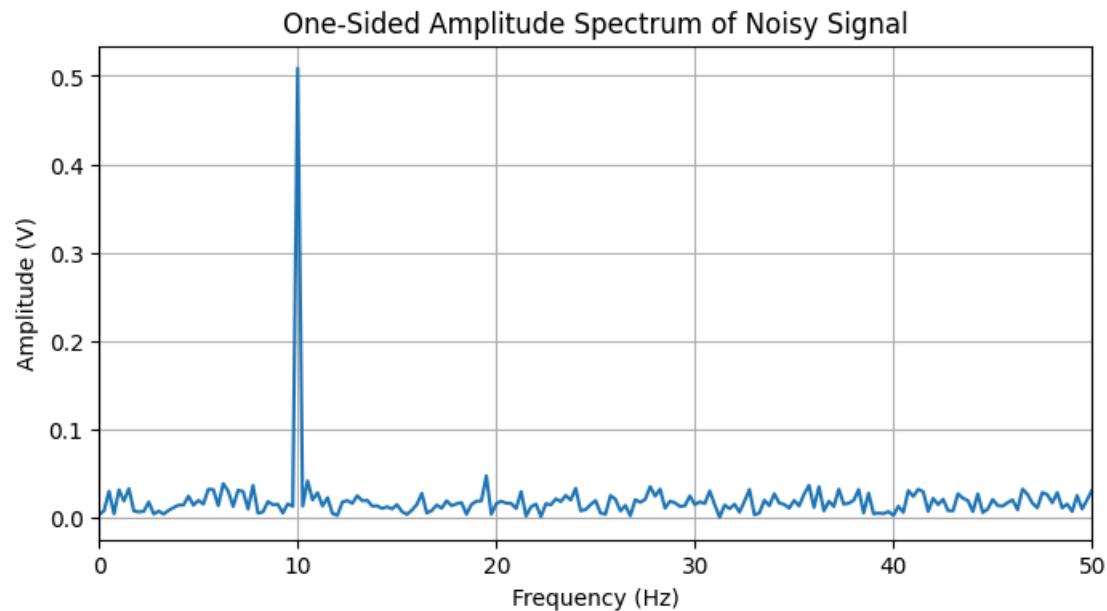
In [63]: R = np.fft.rfft(noisy_signal)
         freqs = np.fft.rfftfreq(N, d=1/fs)

# One-sided amplitude spectrum
R_amp = np.abs(R) / N
R_amp[1:-1] *= 2

plt.figure(figsize=(8,4))
plt.plot(freqs, R_amp)
plt.xlabel("Frequency (Hz)")
plt.ylabel("Amplitude (V)")

```

```
plt.title("One-Sided Amplitude Spectrum of Noisy Signal")
plt.grid(True)
plt.xlim(0, 50)
plt.show()
```



The clean signal has a mean near zero and a variance of approximately 0.125 V^2 . After AWGN is added, the mean remains near zero because the noise is zero-mean, while the variance increases to approximately 0.25 V^2 due to the added noise power. The values are very close to the theoretical results but not exact because the AWGN is generated from a finite set of random samples, causing small variations in the measured statistics.

$$\text{Var}(r) = \text{Var}(x) + \text{Var}(n) = 0.125 \text{ V}^2 + 0.125 \text{ V}^2 = 0.250 \text{ V}^2$$

```
In [38]: # Clean signal statistics
mean_clean = np.mean(x)
var_clean = np.var(x)

# Noisy signal statistics
mean_noisy = np.mean(noisy_signal)
var_noisy = np.var(noisy_signal)

print("Clean Signal")
print(f"Mean      = {mean_clean:.6f}")
print(f"Variance = {var_clean:.6f} V^2")

print("\nNoisy Signal")
print(f"Mean      = {mean_noisy:.6f}")
print(f"Variance = {var_noisy:.6f} V^2")
```

```
Clean Signal
Mean      = -0.000000
Variance = 0.125000 V^2
```

```
Noisy Signal
Mean      = -0.003817
Variance = 0.252740 V^2
```

In the frequency domain, both signals exhibit a peak at 10 Hz, which corresponds to the original sinusoid. The noisy spectrum additionally contains a broadband noise floor across all frequencies because AWGN contains energy distributed over a wide range of frequencies. As a result, noise contributes small frequency components throughout the spectrum rather than at a single frequency. The clean spectrum contains essentially only the 10 Hz component, while the noisy spectrum shows the same 10 Hz peak superimposed on the elevated noise floor. Small variations in the noise floor are expected because the AWGN is generated from a finite set of random samples.

```
In [40]: # Clean spectrum
X = np.fft.rfft(x)
X_amp = np.abs(X) / N
X_amp[1:-1] *= 2

# Noisy spectrum
R = np.fft.rfft(noisy_signal)
R_amp = np.abs(R) / N
R_amp[1:-1] *= 2

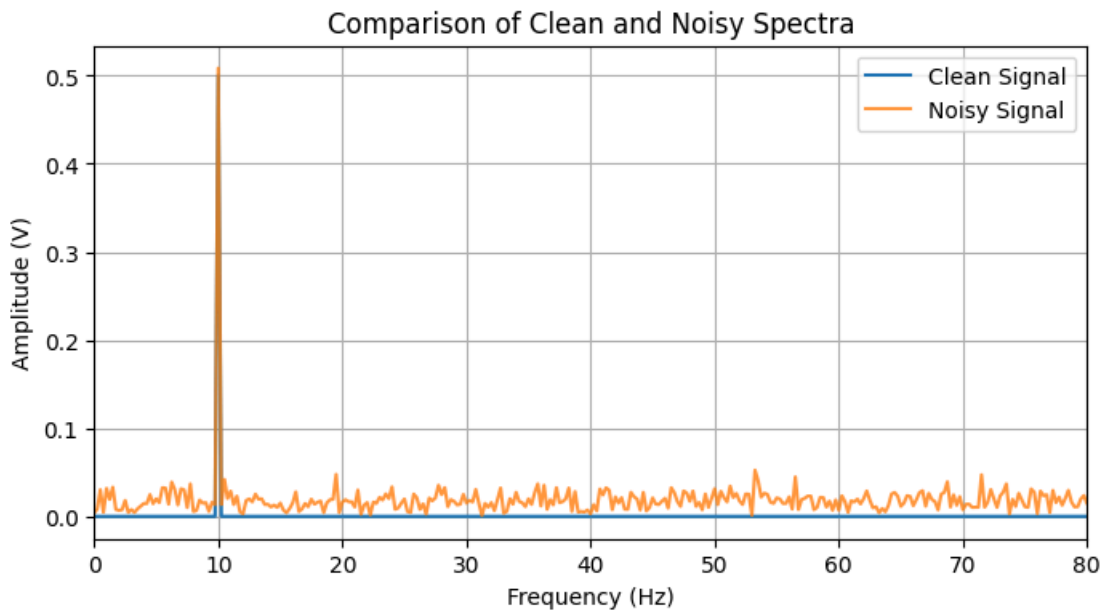
freqs = np.fft.rfftfreq(N, d=1/fs)

plt.figure(figsize=(8,4))

plt.plot(freqs, X_amp, label='Clean Signal')
plt.plot(freqs, R_amp, label='Noisy Signal', alpha=0.8)

plt.xlim(0, 80)
plt.xlabel('Frequency (Hz)')
plt.ylabel('Amplitude (V)')
plt.title('Comparison of Clean and Noisy Spectra')
plt.grid(True)
plt.legend()

plt.show()
```



3.3.1: Different Default RNG Seed

Changing the random seed generates a different set of random noise samples. As a result, the noisy waveform changes, and the measured noise variance, measured SNR, mean, and variance of the noisy signal will be slightly different each time. However, these values should remain close to their theoretical values because the noise is generated with the same target variance. For example, the measured noise variance should stay close to 0.125 V^2 and the measured SNR should remain close to 0 dB, although they may be slightly above or below these values.

```
In [72]: # Signal power of the clean sinusoid
signal_power_v2 = np.mean(x**2)

# 0 dB target SNR
noise_power_v2 = signal_power_v2

# Standard deviation corresponding to the required variance
noise_standard_deviation_v = np.sqrt(noise_power_v2)
```

```

rng = np.random.default_rng(120)

# Generate AWGN realization
noise_samples = noise_standard_deviation_v * rng.standard_normal(N)

# Received signal
noisy_signal = x + noise_samples

# Measured noise variance
measured_noise_variance_v2 = np.var(noise_samples)

# Measured SNR
measured_snr_db = 10 * np.log10(
    np.mean(x**2) / np.mean(noise_samples**2)
)

print(f"Required noise variance: {noise_power_v2:.6f} V^2")
print(f"Measured noise variance: {measured_noise_variance_v2:.6f} V^2")
print(f"Measured SNR: {measured_snr_db:.4f} dB")

rng = np.random.default_rng(7377)

```

```

Required noise variance: 0.125000 V^2
Measured noise variance: 0.123299 V^2
Measured SNR: 0.0583 dB

```

The measured mean and variance of the noisy signal also change slightly because of the different set of random noise samples. The noisy signal mean remains close to zero because the added AWGN is zero-mean, while the variance remains close to the expected value of approximately $0.25 V^2$.

```

In [73]: # Clean signal statistics
mean_clean = np.mean(x)
var_clean = np.var(x)

# Noisy signal statistics
mean_noisy = np.mean(noisy_signal)
var_noisy = np.var(noisy_signal)

print("Clean Signal")
print(f"Mean      = {mean_clean:.6f}")
print(f"Variance = {var_clean:.6f} V^2")

print("\nNoisy Signal")
print(f"Mean      = {mean_noisy:.6f}")
print(f"Variance = {var_noisy:.6f} V^2")

```

```

Clean Signal
Mean      = -0.000000
Variance = 0.125000 V^2

```

```

Noisy Signal
Mean      = -0.005765
Variance = 0.252696 V^2

```

The frequency spectrum also changes slightly because the noise samples are different. The 10 Hz peak remains at the same frequency and nearly the same amplitude since the sinusoidal signal is unchanged, but the shape of the noise floor varies from one realization to another. The noisy spectrum therefore looks slightly different each time even though its overall characteristics remain the same.

```

In [70]: # Clean spectrum
X = np.fft.rfft(x)
X_amp = np.abs(X) / N
X_amp[1:-1] *= 2

# Noisy spectrum
R = np.fft.rfft(noisy_signal)
R_amp = np.abs(R) / N
R_amp[1:-1] *= 2

```

```

freqs = np.fft.rfftfreq(N, d=1/fs)

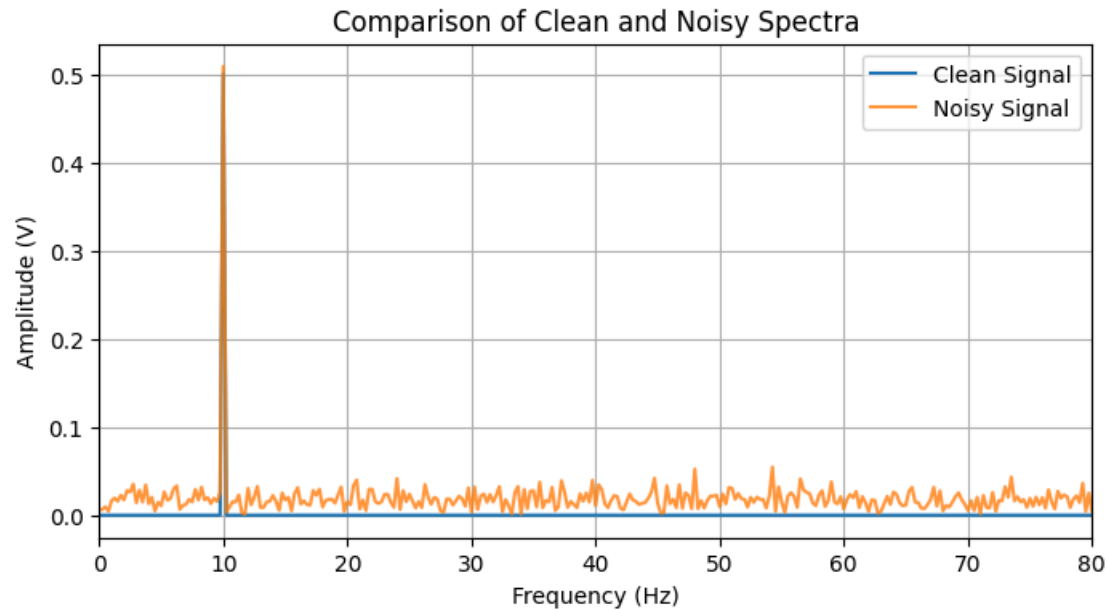
plt.figure(figsize=(8,4))

plt.plot(freqs, X_amp, label='Clean Signal')
plt.plot(freqs, R_amp, label='Noisy Signal', alpha=0.8)

plt.xlim(0, 80)
plt.xlabel('Frequency (Hz)')
plt.ylabel('Amplitude (V)')
plt.title('Comparison of Clean and Noisy Spectra')
plt.grid(True)
plt.legend()

plt.show()

```



```

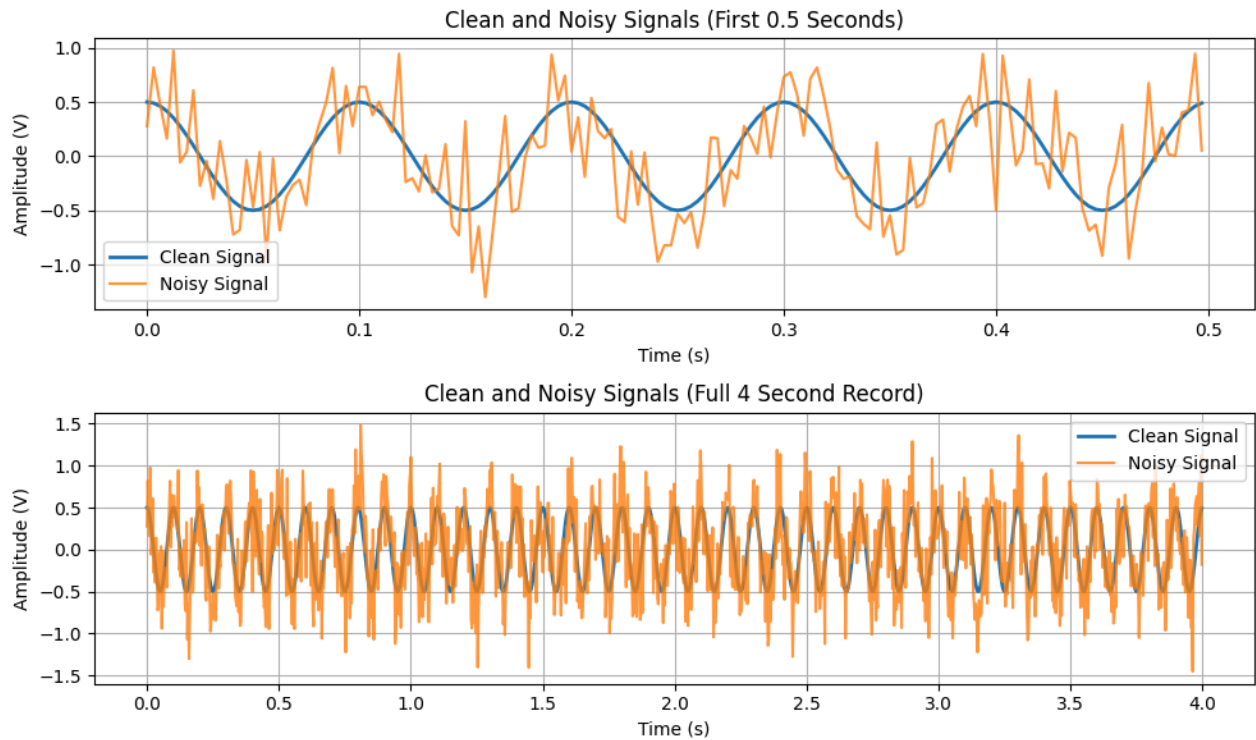
In [68]: plt.figure(figsize=(10, 6))

# First 0.5 s
plt.subplot(2, 1, 1)
plt.plot(t[:160], x[:160], label="Clean Signal", linewidth=2)
plt.plot(t[:160], noisy_signal[:160], label="Noisy Signal", alpha=0.8)
plt.title("Clean and Noisy Signals (First 0.5 Seconds)")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude (V)")
plt.grid(True)
plt.legend()

# Full 4 s record
plt.subplot(2, 1, 2)
plt.plot(t, x, label="Clean Signal", linewidth=2)
plt.plot(t, noisy_signal, label="Noisy Signal", alpha=0.8)
plt.title("Clean and Noisy Signals (Full 4 Second Record)")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude (V)")
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()

```



This behavior can also be seen in the finite noise realization histogram. Changing the seed generates a different finite set of random samples, so the histogram changes shape slightly. Even though each realization follows the same Gaussian distribution, a finite number of samples will not perfectly match the theoretical Gaussian density. As more samples are used, the histogram would more closely match the target Gaussian curve.

```
In [71]: # Voltage axis for theoretical PDF
v = np.linspace(-1.5, 1.5, 1000)

# Target Gaussian PDF
gaussian_pdf = (
    1 / np.sqrt(2 * np.pi * noise_power_v2)
) * np.exp(
    -(v**2) / (2 * noise_power_v2)
)

plt.figure(figsize=(13, 5))

# Histogram of finite noise realization
plt.hist(
    noise_samples,
    bins=36,
    density=True,
    alpha=0.7,
    label="Finite Noise Realization"
)

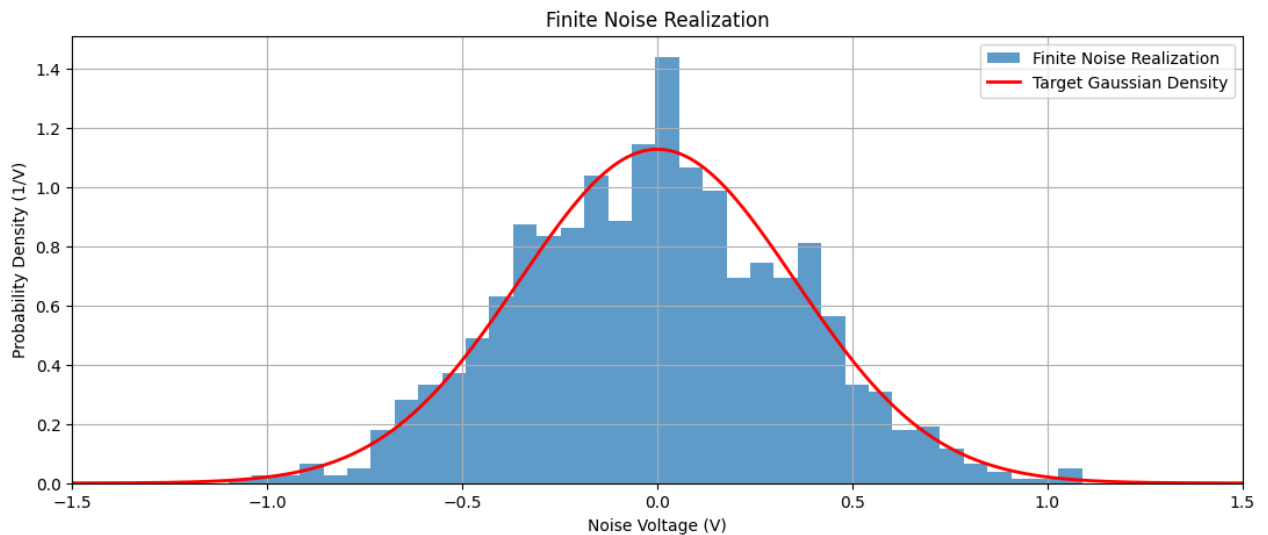
# Theoretical Gaussian PDF
plt.plot(
    v,
    gaussian_pdf,
    'r',
    linewidth=2,
    label="Target Gaussian Density"
)

plt.xlim(-1.5, 1.5)

plt.xlabel("Noise Voltage (V)")
plt.ylabel("Probability Density (1/V)")
plt.title("Finite Noise Realization")
```

```
plt.grid(True)
plt.legend()

plt.show()
```



3.4: Welch PSD estimate

The Welch-bin spacing $\Delta f_{Welch} = \frac{f_s}{N_{FFT}} = \frac{320}{512} = 0.625$ Hz with the number of Welch segment samples N_{FFT} and sampling frequency f_s .

The amplitude change in dB between two amplitudes A_1 and A_2 is $20 \log_{10} \left(\frac{A_2}{A_1} \right)$

Reducing the signal amplitude from 0.5 V to 0.1 V makes the amplitude change $20 \log_{10} \left(\frac{0.1}{0.5} \right) = -13.98$ dB.

The power change in dB between two powers P_2 and P_1 is $10 \log_{10} \left(\frac{P_2}{P_1} \right)$

Reducing the signal amplitude from 0.5 V to 0.1 V makes the power change $20 \log_{10} \left(\frac{0.1^2}{0.5^2} \right) = -13.98$ dB because the power is proportional to the square of the amplitude.

```
In [77]: from scipy import signal

welch_segment_samples = 512
welch_overlap_samples = 256

# Welch frequency-bin spacing
welch_bin_spacing_hz = fs / welch_segment_samples

welch_settings = {
    "fs": fs,
    "window": "hann",
    "nperseg": welch_segment_samples,
    "noverlap": welch_overlap_samples,
    "nfft": welch_segment_samples,
    "scaling": "density",
}

# Clean signal, A = 0.5 V
welch_frequency_hz, clean_psd = signal.welch(
    x,
    **welch_settings
)

# Noisy signal, A = 0.5 V
```

```

_, noisy_psd = signal.welch(
    noisy_signal,
    **welch_settings
)

# Reduced-amplitude signal, A = 0.1 V
x_reduced = 0.1 * np.cos(2 * np.pi * f0 * t)
reduced_noisy_signal = x_reduced + noise_samples

_, reduced_psd = signal.welch(
    reduced_noisy_signal,
    **welch_settings
)

# Amplitude and power changes
amplitude_change_db = 20 * np.log10(0.1 / 0.5)
power_change_db = 10 * np.log10((0.1**2) / (0.5**2))

print(f"Welch-bin spacing: {welch_bin_spacing_hz:.3f} Hz")
print(f"Amplitude change: {amplitude_change_db:.3f} dB")
print(f"Power change: {power_change_db:.3f} dB")

```

```

Welch-bin spacing: 0.625 Hz
Amplitude change: -13.979 dB
Power change: -13.979 dB

```

The Welch PSD estimate shows a strong spectral line near 10 Hz, which corresponds to the sinusoidal signal. This peak appears because most of the signal's power is concentrated at a single frequency. In the clean PSD, nearly all of the power is contained in this spectral line, with very little power at other frequencies.

The noisy PSD contains the same 10 Hz spectral line, but it also includes a broadband noise floor caused by the added AWGN. Unlike the sinusoid, which has power concentrated at one frequency, AWGN spreads its power across a wide range of frequencies. This causes the PSD to show nonzero power over much of the frequency spectrum.

The noise floor is not perfectly flat and shows small fluctuations from one frequency bin to another. This occurs because the Welch PSD is only an estimate based on a finite number of samples and overlapping segments. Since a finite realization of random noise is used, the estimated noise power varies slightly across frequency bins. These random fluctuations are known as finite-estimate variation and would decrease if a longer data record were used.

```

In [80]: epsilon = 1e-35

plt.figure(figsize=(8,5))

plt.plot(
    welch_frequency_hz,
    10*np.log10(np.maximum(clean_psd, epsilon)),
    label='Clean Signal (A = 0.5 V)'
)

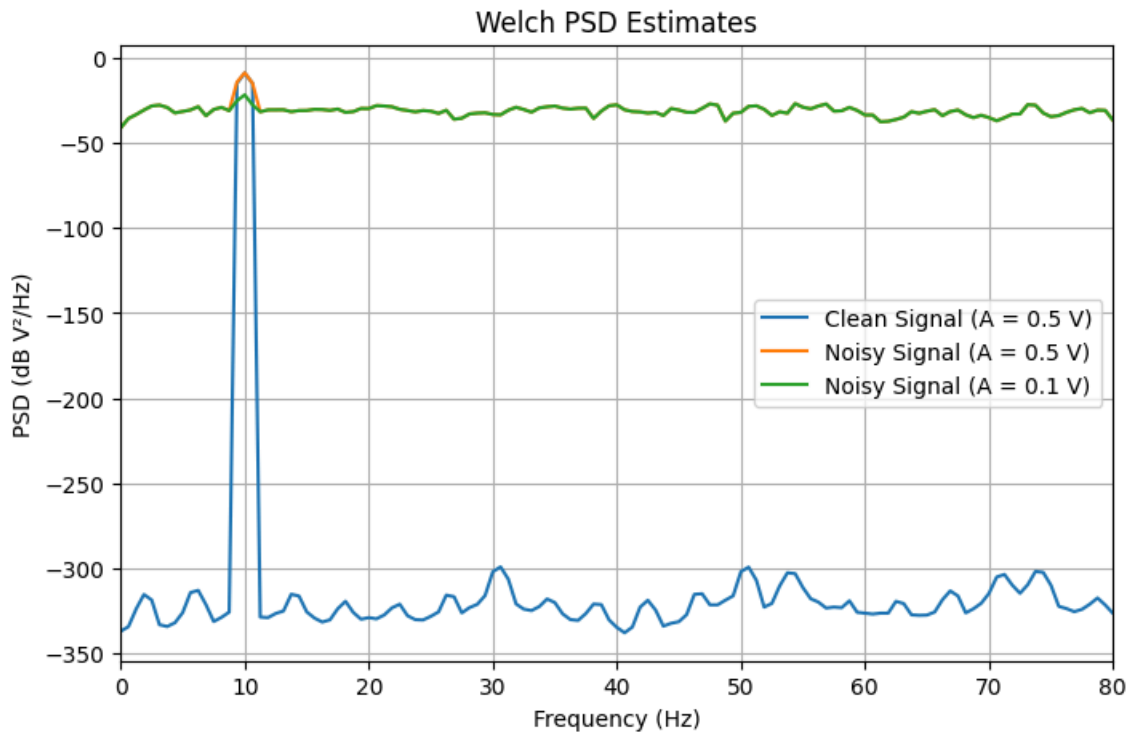
plt.plot(
    welch_frequency_hz,
    10*np.log10(np.maximum(noisy_psd, epsilon)),
    label='Noisy Signal (A = 0.5 V)'
)

plt.plot(
    welch_frequency_hz,
    10*np.log10(np.maximum(reduced_psd, epsilon)),
    label='Noisy Signal (A = 0.1 V)'
)

plt.xlabel('Frequency (Hz)')
plt.ylabel('PSD (dB V2/Hz)')
plt.title('Welch PSD Estimates')
plt.xlim(0, 80)
plt.grid(True)
plt.legend()

```

```
plt.show()
```



4. Task 2: Sampling Aliasing, and Filtering

4.1: Rectangular pulse

The first two positive theoretical zero crossings should occur at $f = \frac{1}{T_p}, \frac{2}{T_p}$ because zero crossings occur when $fT_p = \pm 1, \pm 2, \pm 3$. Since $T_p = 1$ second, the first two positive zeros should theoretically occur at $f = 1$ Hz and $f = 2$ Hz.

The graph below lists the sample rate $f_s = \frac{L}{T_p}$ and Nyquist Frequency $f_N = \frac{f_s}{2}$.

```
In [111]: # Parameters supplied by the manual
pulse_width_s = 1.0
observation_s = 4.0
samples_per_pulse_values = [1, 2, 4, 8, 16, 32]

# Large FFT size to approximate the DTFT
fft_size = 16384

first_zero_hz = 1 / pulse_width_s
second_zero_hz = 2 / pulse_width_s

rectangular_records = []

for samples_per_pulse in samples_per_pulse_values:

    sample_rate_sps = samples_per_pulse / pulse_width_s

    nyquist_hz = sample_rate_sps / 2

    sample_interval_s = 1 / sample_rate_sps

    # Observe pulse in [-2, 2)
    time_s = np.arange(
        -observation_s / 2,
```

```

        observation_s / 2,
        sample_interval_s
    )

    # Rectangular pulse of width 1 second
    pulse = (np.abs(time_s) < pulse_width_s / 2).astype(float)

    # Centered zero-padded FFT
    sampled_spectrum = np.fft.fftshift(
        np.fft.fft(pulse, fft_size)
    )

    frequency_hz = np.fft.fftshift(
        np.fft.fftfreq(
            fft_size,
            d=sample_interval_s
        )
    )

    sampled_magnitude = np.abs(sampled_spectrum)
    sampled_magnitude /= np.max(sampled_magnitude)

    theoretical_magnitude = np.abs(
        np.sinc(frequency_hz * pulse_width_s)
    )

    rectangular_records.append({
        "samples_per_pulse": samples_per_pulse,
        "sample_rate_sps": sample_rate_sps,
        "nyquist_hz": nyquist_hz,
        "time_s": time_s,
        "pulse": pulse,
        "frequency_hz": frequency_hz,
        "sampled_magnitude": sampled_magnitude,
        "theoretical_magnitude": theoretical_magnitude,
    })

print(f"First two positive sinc zeros: {first_zero_hz:g}, {second_zero_hz:g} Hz")

for record in rectangular_records:
    print(
        f"L={record['samples_per_pulse']:2d}: "
        f"fs={record['sample_rate_sps']:5.1f} Hz, "
        f"Nyquist={record['nyquist_hz']:4.1f} Hz"
    )

```

```

First two positive sinc zeros: 1, 2 Hz
L= 1: fs=  1.0 Hz, Nyquist= 0.5 Hz
L= 2: fs=  2.0 Hz, Nyquist= 1.0 Hz
L= 4: fs=  4.0 Hz, Nyquist= 2.0 Hz
L= 8: fs=  8.0 Hz, Nyquist= 4.0 Hz
L=16: fs= 16.0 Hz, Nyquist= 8.0 Hz
L=32: fs= 32.0 Hz, Nyquist=16.0 Hz

```

For small values of L , the sampling rate and Nyquist frequency are too low to capture all of these zeros. For example, when $L=1$, the Nyquist frequency is only 0.5 Hz, so even the first theoretical zero at 1 Hz cannot be observed. As L increases, the Nyquist frequency increases and more of the sinc spectrum becomes visible, allowing additional zero crossings to be observed.

Increasing L improves the sampled representation because more samples are used to describe the pulse in time. This produces a more accurate approximation of the rectangular shape and a smoother FFT. However, the physical pulse width remains unchanged at $T_p = 1$ second. Increasing L only changes the sampling density, not the actual duration of the pulse.

```

In [112]: import matplotlib.pyplot as plt

fig, axes = plt.subplots(
    3, 2,

```

```

    figsize=(10, 12),
    sharex=True
)

for ax, record in zip(
    axes.ravel(),
    rectangular_records
):

    ax.stem(
        record["time_s"],
        record["pulse"],
        basefmt=" "
    )

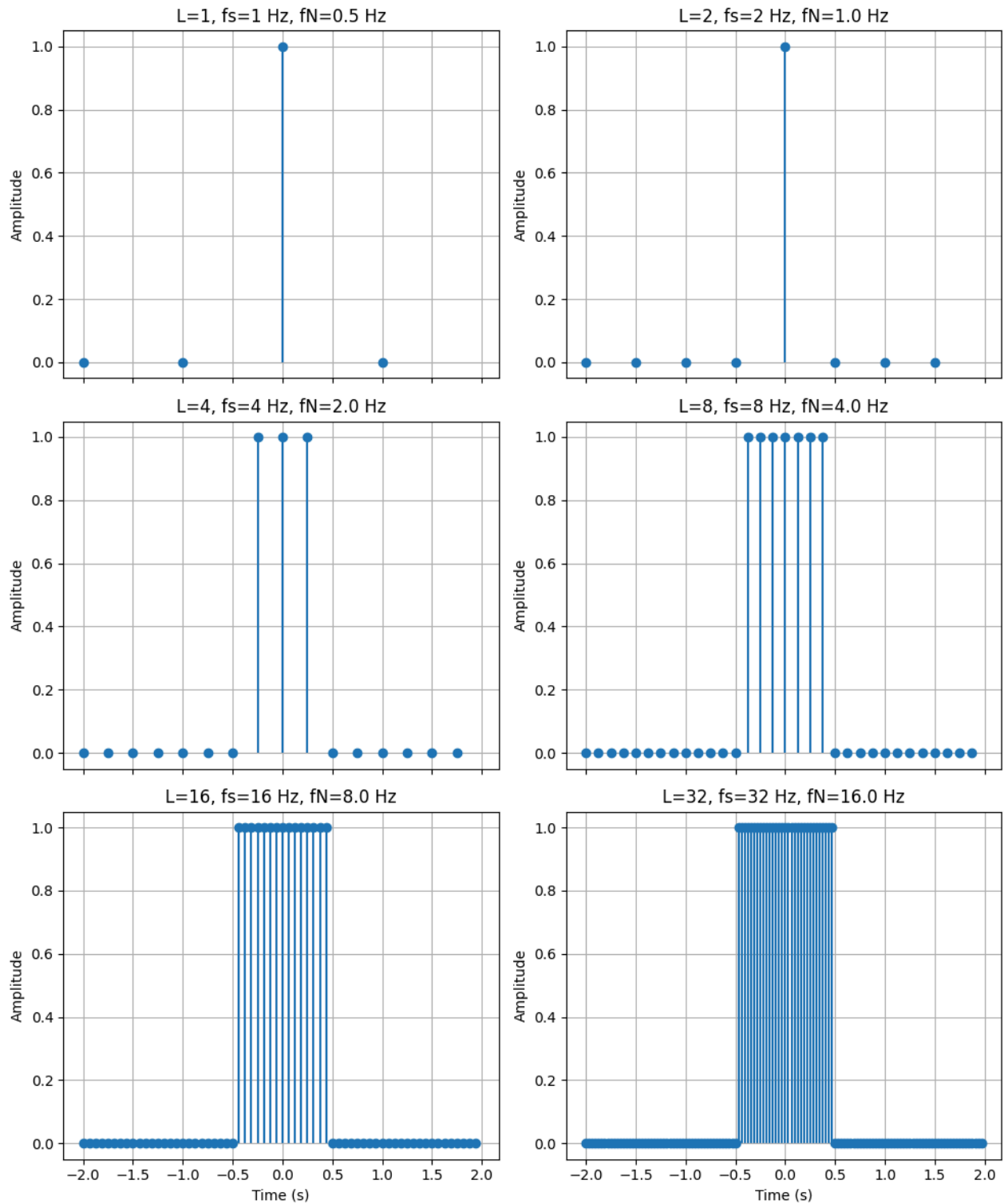
    ax.set_title(
        f"L={record['samples_per_pulse']}, "
        f"fs={record['sample_rate_sps']:.0f} Hz, "
        f"N={record['nyquist_hz']:.1f} Hz"
    )

    ax.set_ylabel("Amplitude")
    ax.set_ylim(-0.05, 1.05)
    ax.grid(True)

axes[-1,0].set_xlabel("Time (s)")
axes[-1,1].set_xlabel("Time (s)")

plt.tight_layout()
plt.show()

```



```
In [116... fig, axes = plt.subplots(3, 2, figsize=(10, 12))

for ax, record in zip(axes.ravel(), rectangular_records):

    fN = record["nyquist_hz"]

    ax.plot(
        record["frequency_hz"],
        record["sampled_magnitude"],
        label="sampled pulse",
        linewidth=2
    )
```

```

ax.plot(
    record["frequency_hz"],
    record["theoretical_magnitude"],
    '--',
    label=r'$|\mathrm{sinc}(fT_p)|$',
    linewidth=1.5
)

if record["samples_per_pulse"] >= 16:
    ax.set_xlim(-5, 5)
else:
    ax.set_xlim(-fN, fN)
ax.set_ylim(-0.02, 1.05)

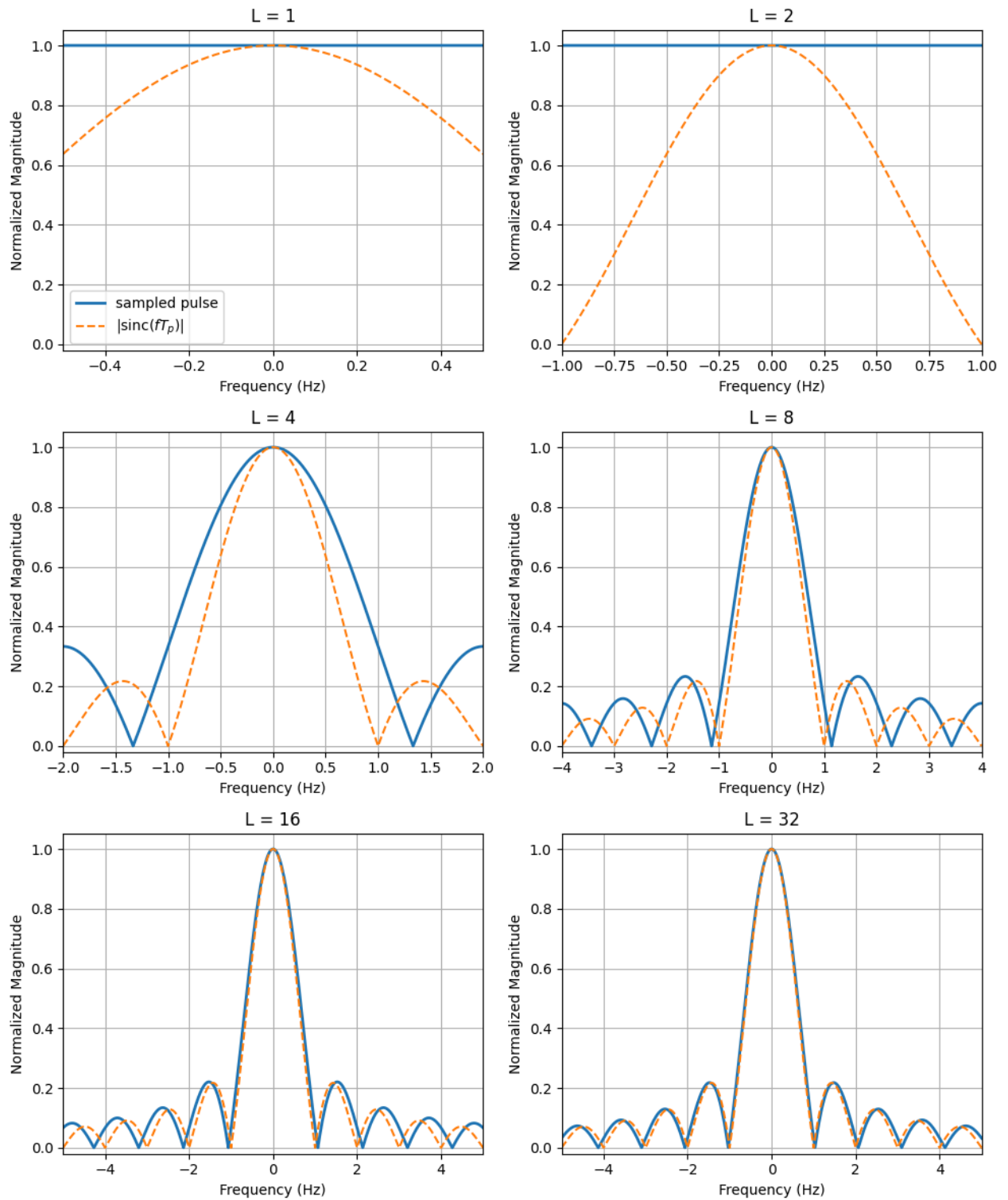
ax.set_title(
    f"L = {record['samples_per_pulse']}"
)

ax.set_xlabel("Frequency (Hz)")
ax.set_ylabel("Normalized Magnitude")
ax.grid(True)

if record["samples_per_pulse"] == 1:
    ax.legend()

plt.tight_layout()
plt.show()

```



4.2: Sinusoidal signals

The time duration of the signal $T = \frac{\text{Number of Cycles}}{f_0} = \frac{5}{10000 \text{ Hz}} = 0.0005 \text{ seconds}$.

The sample count $N = f_s T = (0.0005) f_s$.

The Nyquist Frequency $f_N = \frac{f_s}{2}$.

The predicted principal alias frequency was calculated using $f_{\text{alias}} = \left[\left(f_0 + \frac{f_s}{2} \right) \bmod f_s \right] - \frac{f_s}{2}$

Case 1: $f_s = 50f_0$

$$f_s = 50(10 \text{ kHz}) = 500 \text{ kHz}$$

$$f_{\text{alias}} = [(10 + 250) \bmod 500] - 250 = 260 - 250 = 10 \text{ kHz}$$

$$f_N = \frac{f_s}{2} = \frac{500}{2} = 250 \text{ kHz}$$

$$N = f_s T = (500000)(0.0005) = 250$$

Case 2: $f_s = 8f_0$

$$f_s = 8(10 \text{ kHz}) = 80 \text{ kHz}$$

$$f_{\text{alias}} = [(10 + 40) \bmod 80] - 40 = 50 - 40 = 10 \text{ kHz}$$

$$f_N = \frac{80}{2} = 40 \text{ kHz}$$

$$N = (80000)(0.0005) = 40$$

Case 3: $f_s = 4f_0$

$$f_s = 4(10 \text{ kHz}) = 40 \text{ kHz}$$

$$f_{\text{alias}} = [(10 + 20) \bmod 40] - 20 = 30 - 20 = 10 \text{ kHz}$$

$$f_N = \frac{40}{2} = 20 \text{ kHz}$$

$$N = (40000)(0.0005) = 20$$

Case 4: $f_s = 2f_0$

$$f_s = 2(10 \text{ kHz}) = 20 \text{ kHz}$$

$$f_{\text{alias}} = [(10 + 10) \bmod 20] - 10 = 0 - 10 = -10 \text{ kHz}$$

$$f_N = \frac{20}{2} = 10 \text{ kHz}$$

$$N = (20000)(0.0005) = 10$$

The signal lies exactly at the Nyquist frequency because the sampling frequency is exactly twice the signal frequency. With zero-phase sampling, the cosine is sampled at alternating peaks and valleys, alternating between $+1$ and -1 . This result depends strongly on the sampling phase. If the sampling instants were shifted by a fraction of a period, a completely different sequence would be obtained. For example, sampling a quarter-period later would produce samples near zero. Because there are only two samples per cycle, the sampled signal does not uniquely determine the original continuous-time waveform. Small changes in sampling phase can therefore lead to very different sample sequences, even though the underlying sinusoid is the same. This sensitivity occurs because the signal is being sampled exactly at the Nyquist limit.

Case 5: $f_s = f_0$

$$f_s = 10 \text{ kHz}$$

$$f_{\text{alias}} = [(10 + 5) \bmod 10] - 5 = 5 - 5 = 0 \text{ kHz}$$

$$f_N = \frac{10}{2} = 5 \text{ kHz}$$

$$N = (10000)(0.0005) = 5$$

The sampling frequency is the same as the signal frequency, so one sample is taken every cycle of the cosine. This means every sample is taken at the same point in the waveform, so all samples have the same value. Changing the phase changes what that constant value is, but the sequence remains constant. Because a constant sequence does not vary with time, its FFT contains only a DC component. Therefore, the original sinusoid aliases to 0 Hz regardless of the sampling phase.

```
In [118... # Parameters supplied by the manual
tone_hz = 10_000.0
cycles = 5
sample_rate_ratios = [50, 8, 4, 2, 1]

duration_s = cycles / tone_hz

reference_time_s = np.linspace(
    0,
    duration_s,
    3000,
    endpoint=False
)

reference_signal = np.cos(
    2 * np.pi * tone_hz * reference_time_s
)

sampling_records = []

for ratio in sample_rate_ratios:

    sample_rate_sps = ratio * tone_hz

    sample_count = int(
        np.round(duration_s * sample_rate_sps)
    )

    nyquist_hz = sample_rate_sps / 2

    time_s = np.arange(sample_count) / sample_rate_sps

    # Zero-phase cosine
    samples = np.cos(
        2 * np.pi * tone_hz * time_s
    )

    # Principal alias formula from the manual
    alias_hz = (
        ((tone_hz + sample_rate_sps / 2) % sample_rate_sps)
        - sample_rate_sps / 2
    )

    frequency_hz = np.fft.fftshift(
        np.fft.fftfreq(
            sample_count,
            d=1/sample_rate_sps
        )
    )

    spectrum = np.fft.fftshift(
        np.fft.fft(samples)
    )

    sampling_records.append({
```

```

        "ratio": ratio,
        "sample_rate_sps": sample_rate_sps,
        "sample_count": sample_count,
        "nyquist_hz": nyquist_hz,
        "time_s": time_s,
        "samples": samples,
        "alias_hz": alias_hz,
        "frequency_hz": frequency_hz,
        "spectrum": spectrum,
    })

for record in sampling_records:
    print(
        f"fs/f0={record['ratio']:2d}: "
        f"N={record['sample_count']:3d}, "
        f"Nyquist={record['nyquist_hz']/1e3:6.1f} kHz, "
        f"alias={record['alias_hz']/1e3:6.1f} kHz"
    )

```

```

fs/f0=50: N=250, Nyquist= 250.0 kHz, alias=  10.0 kHz
fs/f0= 8: N= 40, Nyquist=  40.0 kHz, alias=  10.0 kHz
fs/f0= 4: N= 20, Nyquist=  20.0 kHz, alias=  10.0 kHz
fs/f0= 2: N= 10, Nyquist=  10.0 kHz, alias= -10.0 kHz
fs/f0= 1: N=  5, Nyquist=   5.0 kHz, alias=   0.0 kHz

```

```

In [123... fig, axes = plt.subplots(
    len(sampling_records),
    1,
    figsize=(10, 14)
)

for ax, record in zip(axes, sampling_records):

    ax.plot(
        reference_time_s * 1e3,
        reference_signal,
        'k',
        linewidth=1,
        label='continuous reference'
    )

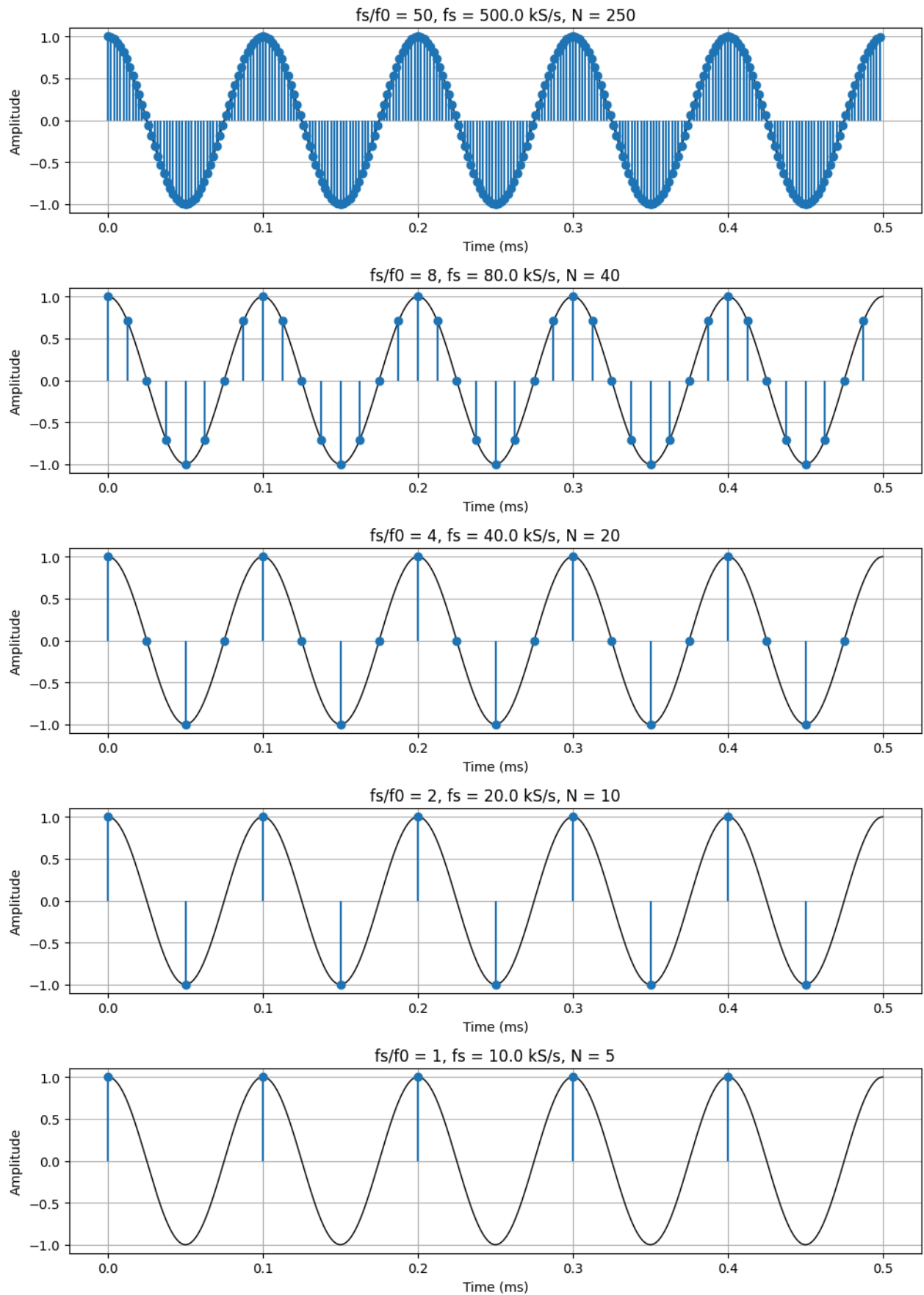
    ax.stem(
        record["time_s"] * 1e3,
        record["samples"],
        linefmt='C0-',
        markerfmt='C0o',
        basefmt=' '
    )

    ax.set_title(
        f"fs/f0 = {record['ratio']}, "
        f"fs = {record['sample_rate_sps']/1e3:.1f} kS/s, "
        f"N = {record['sample_count']}"
    )

    ax.set_xlabel("Time (ms)")
    ax.set_ylabel("Amplitude")
    ax.grid(True)

plt.tight_layout()
plt.show()

```



```
In [125... fig, axes = plt.subplots(
    3, 2,
    figsize=(12, 10)
)
```

```

axes = axes.ravel()

for ax, record in zip(axes[:5], sampling_records):

    magnitude = np.abs(record["spectrum"])

    if np.max(magnitude) > 0:
        magnitude /= np.max(magnitude)

    # Stem plot (spikes + dots)
    markerline, stemlines, baseline = ax.stem(
        record["frequency_hz"] / 1e3,
        magnitude,
        baselfmt=" "
    )

    plt.setp(markerline, markersize=5)

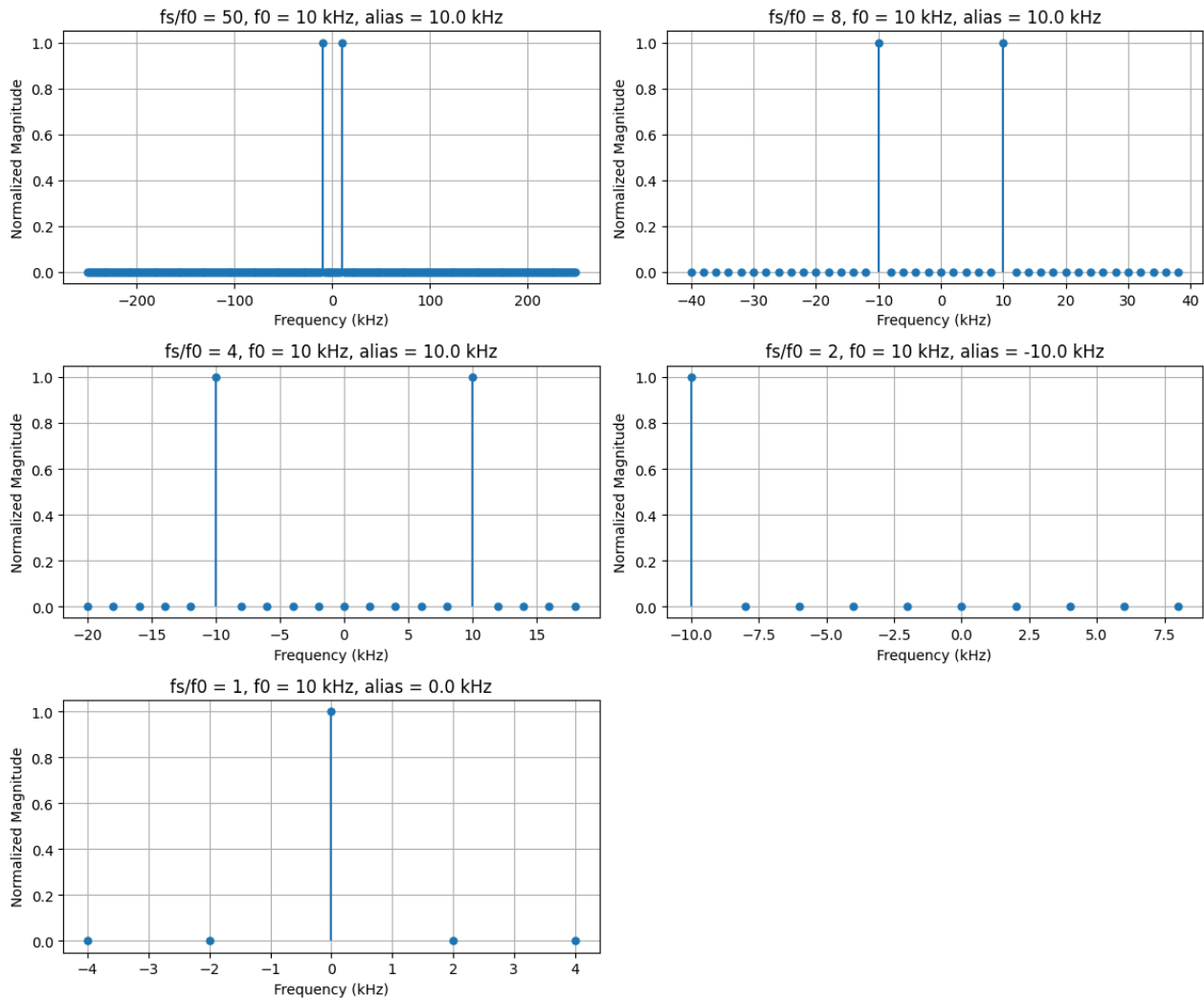
    ax.set_title(
        f"fs/f0 = {record['ratio']}, "
        f"f0 = {tone_hz/1e3:.0f} kHz, "
        f"alias = {record['alias_hz']/1e3:.1f} kHz"
    )

    ax.set_xlabel("Frequency (kHz)")
    ax.set_ylabel("Normalized Magnitude")
    ax.grid(True)

# Remove unused sixth subplot
fig.delaxes(axes[5])

plt.tight_layout()
plt.show()

```



4.3: Aliasing and filters

A practical FIR filter cannot switch instantly from passing frequencies to blocking frequencies. Instead, there is a finite transition band where the filter gradually changes from the passband to the stopband. In this experiment, the cutoff frequency is 250 Hz, so frequencies well below 250 Hz, such as 100 Hz and 200 Hz, pass through with little attenuation, while frequencies above the cutoff, such as 300 Hz, are increasingly attenuated. The gradual change occurs because the filter has a finite number of taps (401), which limits how sharp the cutoff can be.

A digital filter cannot remove aliasing that has already occurred during analog-to-digital conversion because multiple analog frequencies can map to the same discrete frequency after sampling. Once this happens, the ADC stores only the aliased frequency and no longer retains information about the original frequency component. As a result, different analog signals can produce the same sampled signal, making it impossible for a digital filter to determine which frequency was actually present before sampling. Since the original information has already been lost, digital processing can remove unwanted frequencies but cannot recover the true pre-aliased signal.

To prevent aliasing, an analog anti-alias filter must be placed before the ADC in a physical receiver. The analog filter removes frequency components above the Nyquist frequency before sampling occurs, preventing them from folding into the desired frequency range. Once the ADC has sampled the signal, it is too late to remove aliasing with a digital filter.

```
In [140... # Parameters supplied by the manual
sample_rate_sps = 10_000.0
duration_s = 2.0
tone_frequencies_hz = [100.0, 200.0, 300.0]
```

```

noise_standard_deviation_v = 0.1
cutoff_hz = 250.0
tap_count = 401

#
# The plotting cell uses: filter_frequency_hz, input_amplitude,
# response_frequency_hz, response, output_amplitude, cutoff_hz, and tap_count.
#

sample_count = int(sample_rate_sps * duration_s)

time_s = np.arange(sample_count) / sample_rate_sps

# Clean signal: sum of three unit-amplitude cosines
clean_signal = (
    np.cos(2 * np.pi * tone_frequencies_hz[0] * time_s)
    + np.cos(2 * np.pi * tone_frequencies_hz[1] * time_s)
    + np.cos(2 * np.pi * tone_frequencies_hz[2] * time_s)
)

rng = np.random.default_rng(2026)

noise_samples = (
    noise_standard_deviation_v
    * rng.standard_normal(sample_count)
)

noisy_signal = clean_signal + noise_samples

# FIR filter
taps = signal.firwin(
    tap_count,
    cutoff_hz,
    fs=sample_rate_sps,
    window="hann"
)

filtered_signal = signal.filtfilt(
    taps,
    [1.0],
    noisy_signal
)

# Frequency axis for spectra
filter_frequency_hz = np.fft.rfftfreq(
    sample_count,
    d=1/sample_rate_sps
)

# Input spectrum
input_fft = np.fft.rfft(noisy_signal)
input_amplitude = np.abs(input_fft) / sample_count
input_amplitude[1:-1] *= 2

# Output spectrum
output_fft = np.fft.rfft(filtered_signal)
output_amplitude = np.abs(output_fft) / sample_count
output_amplitude[1:-1] *= 2

# Filter response
response_frequency_hz, response = signal.freqz(
    taps,
    worN=8192,
    fs=sample_rate_sps
)

tone_measurements = {}

for tone_frequency_hz in tone_frequencies_hz:

```

```

tone_index = np.argmin(
    np.abs(filter_frequency_hz - tone_frequency_hz)
)

change_db = 20 * np.log10(output_amplitude[tone_index]/input_amplitude[tone_index])

tone_measurements[tone_frequency_hz] = (
    input_amplitude[tone_index],
    output_amplitude[tone_index],
    change_db
)

print(f"{tone_frequency_hz:3.0f} Hz: "
      f"input={input_amplitude[tone_index]:.6f} V, "
      f"output={output_amplitude[tone_index]:.6f} V, "
      f"change={change_db:.4f} dB")

```

100 Hz: input=0.998499 V, output=0.998417 V, change=-0.0007 dB
 200 Hz: input=0.999624 V, output=1.011716 V, change=0.1044 dB
 300 Hz: input=0.999261 V, output=0.001686 V, change=-55.4542 dB

The change in amplitude for each tone is calculated using $20 \log_{10} \left(\frac{A_{\text{out}}}{A_{\text{in}}} \right)$ with input amplitude A_{in} measured from the FFT and output amplitude A_{out} measured from the FFT after filtering. The input and output amplitudes aren't exactly 1V because of the Gaussian noise added.

100 Hz Tone

$$\text{Change (dB)} = 20 \log_{10} \left(\frac{A_{\text{out}}}{A_{\text{in}}} \right) = 20 \log_{10} \left(\frac{0.998417}{0.998499} \right) = -0.0007 \text{ dB}$$

200 Hz Tone

$$\text{Change (dB)} = 20 \log_{10} \left(\frac{A_{\text{out}}}{A_{\text{in}}} \right) = 20 \log_{10} \left(\frac{1.011716}{0.999624} \right) = 0.1044 \text{ dB}$$

300 Hz Tone

$$\text{Change (dB)} = 20 \log_{10} \left(\frac{A_{\text{out}}}{A_{\text{in}}} \right) = 20 \log_{10} \left(\frac{0.001686}{0.999261} \right) = -55.4542 \text{ dB}$$

```

In [139... plt.figure(figsize=(10, 8))

# Input spectrum
plt.subplot(3, 1, 1)
plt.plot(filter_frequency_hz, input_amplitude)
plt.xlim(0, 500)
plt.title("Input Amplitude Spectrum")
plt.ylabel("Amplitude (V)")
plt.grid(True)

# Filter response
plt.subplot(3, 1, 2)

plt.plot(
    response_frequency_hz,
    20*np.log10(np.maximum(np.abs(response), 1e-12))
)

# Red dotted cutoff line
plt.axvline(
    x=cutoff_hz,
    color='red',
    linestyle='--',

```

```

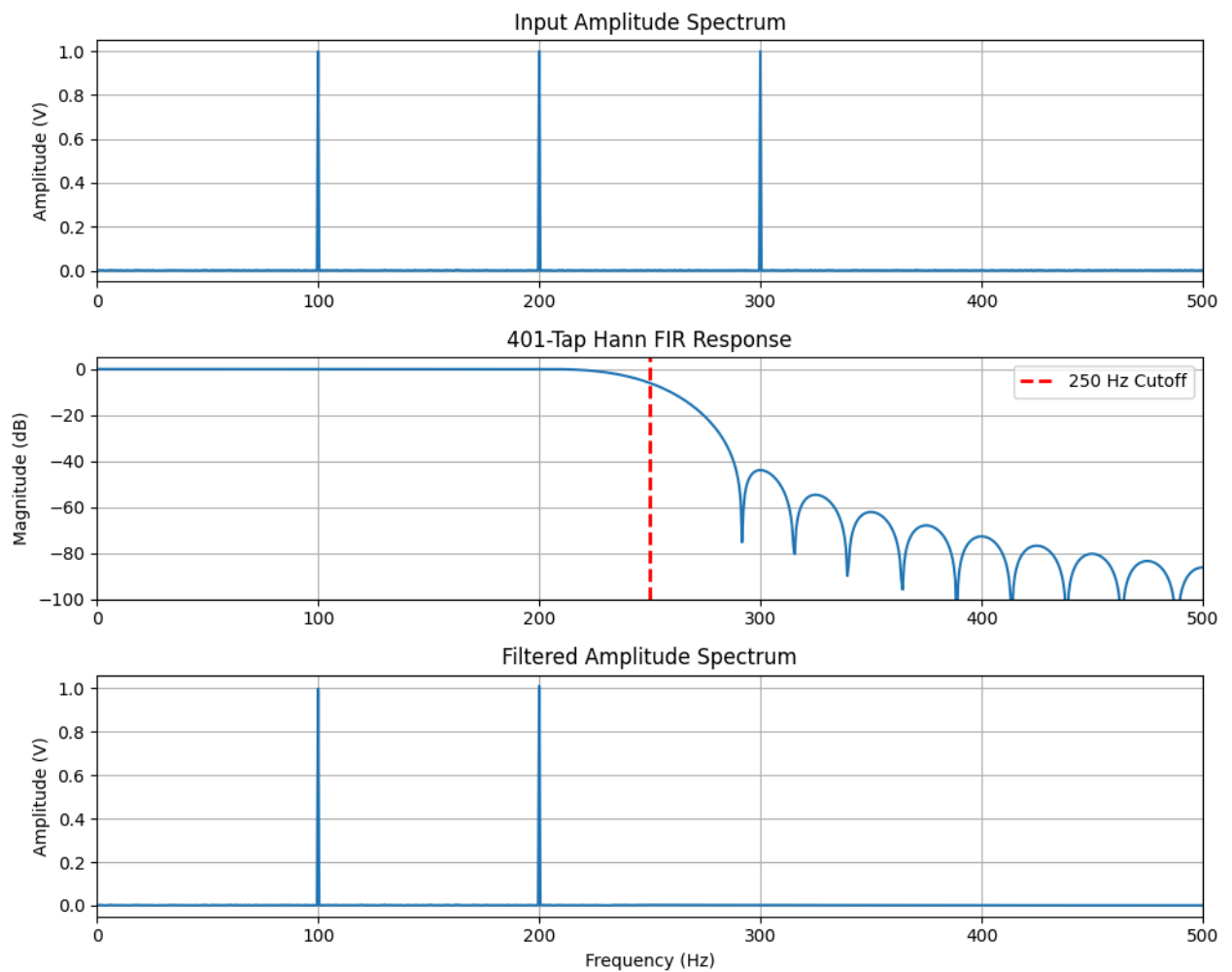
        linewidth=2,
        label='250 Hz Cutoff'
    )

plt.xlim(0, 500)
plt.ylim(-100,5)
plt.title(f"{tap_count}-Tap Hann FIR Response")
plt.ylabel("Magnitude (dB)")
plt.legend()
plt.grid(True)

# Output spectrum
plt.subplot(3, 1, 3)
plt.plot(filter_frequency_hz, output_amplitude)
plt.xlim(0, 500)
plt.title("Filtered Amplitude Spectrum")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Amplitude (V)")
plt.grid(True)

plt.tight_layout()
plt.show()

```

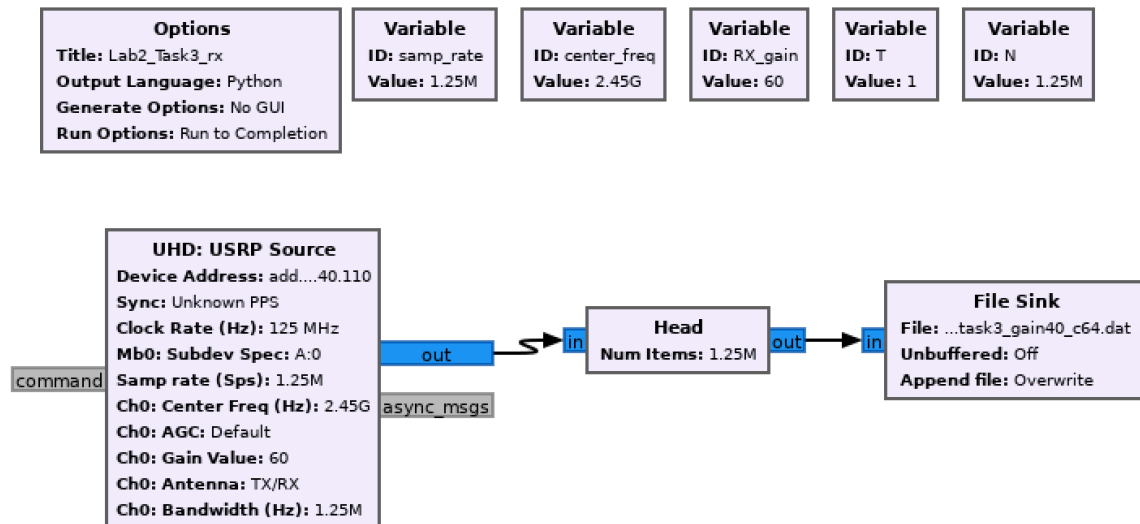


5. Task 3: In-Lab Signal Measurement with USRP N310s

5.1: GNU Radio capture procedure

Questions 1 and 2

Receiver flowgraph



```
[INFO] [UHD] linux; GNU C++ version 13.3.0; Boost_108600; UHD_4.8.0.HEAD-release
[INFO] [MPMD] Initializing 1 device(s) in parallel with args: mgmt_addr=10.61.40.110,type=n3xx,product=n310,serial=3176DD6,name=ni-n3xx-3176DD6,fpga=HG,claimed=False,addr=10.61.40.110
[INFO] [MPM.PeriphManager] init() called with device args `fpga=HG,mgmt_addr=10.61.40.110,name=ni-n3xx-3176DD6,product=n310,clock_source=internal,time_source=internal'.
[INFO] [MULTI_USRP] 1) catch time transition at pps edge
[INFO] [MULTI_USRP] 2) set times next pps (synchronously)
[INFO] [0/Radio#0] Re-initializing dboard to apply bandwidth settings. This may take some time.
[INFO] [0/Radio#0] Bandwidth settings applied: 1250000.000000
Press Enter to quit: 
```

The **noise capture sample count** $N = \text{samp_rate} \times T = 1.25 \text{ MS/s} \times 1.0 \text{ s} = 1.25 \text{ MegaSamples}$.

The complex64 file should contain $8N$ bytes $= 8 * 1.25 * 10^6 = 10,000,000$ bytes

```
In [2]: 8 * 1_250_000
```

```
Out[2]: 10000000
```

We can verify that work/task3_noise_c64.dat is of size 10,000,000 bytes with a `ls -l`.

```
In [4]: ls -l work | grep task3_noise_c6*
```

```
-rw-r--r-- 1 marcpham staff 10000000 Sep 12 09:39 task3_noise_c64.dat
```

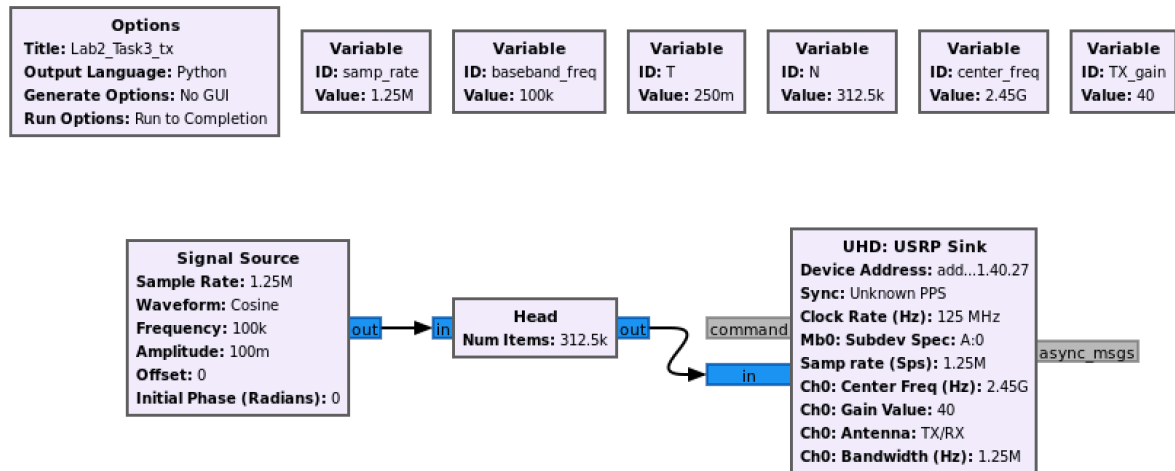
Question 3

The transmitter requires a Head block because the Signal Source generates samples continuously. Without the Head block, the transmitter would transmit indefinitely. The Head block limits the output to exactly 312,500 samples, which corresponds to the required 0.25-second transmission at a sample rate of 1.25 MS/s. This ensures that each experiment uses the same finite-length signal and allows the transmission to terminate automatically.

The receiver is started before the transmitter to ensure that the entire transmitted burst is captured. Since the transmitted tone lasts only 0.25 seconds, starting the transmitter before the receiver could cause the beginning of the burst to be missed. By starting the receiver first, all transmitted samples are recorded within the 8-second capture window, enabling accurate power, frequency, and SNR measurements.

The **transmit sample count** $N = \text{samp_rate} \times T = 1.25 \text{ MS/s} \times 0.25 \text{ s} = 312.5 \text{ kiloSamples}$ where the transmit duration T was specified as 0.25 seconds.

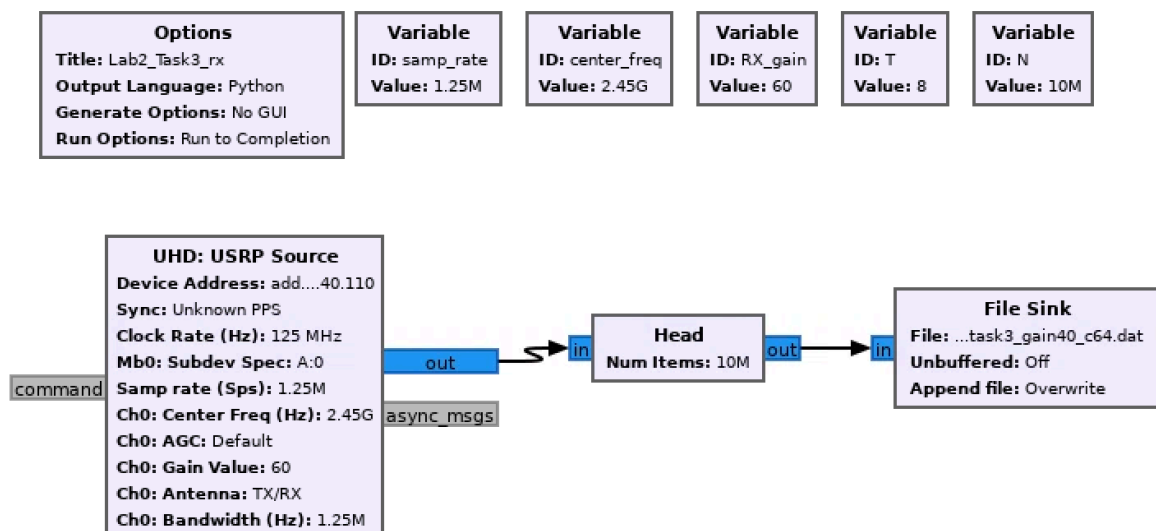
Transmitter flowgraph



```
[INFO] [UHD] linux; GNU C++ version 13.3.0; Boost_108600; UHD_4.8.0.HEAD-release
[INFO] [MPMD] Initializing 1 device(s) in parallel with args: mgmt addr=10.61.40.27,type=n3xx,p
product=n310,serial=3176DE1,name=ni-n3xx-3176DE1,fpga=HG,claimed=False,addr=10.61.40.27
[INFO] [MPM.PeriphManager] init() called with device args `fpga=HG,mgmt addr=10.61.40.27,name=n
i-n3xx-3176DE1,product=n310,clock_source=internal,time_source=internal'.
[INFO] [MULTI_USRP] 1) catch time transition at pps edge
[INFO] [MULTI_USRP] 2) set times next pps (synchronously)
[INFO] [0/Radio#0] Re-initializing dboard to apply bandwidth settings. This may take some time.
[INFO] [0/Radio#0] Bandwidth settings applied: 1250000.000000
[WARNING] [0/Radio#0] Attempting to set tick rate to 0. Skipping.
Press Enter to quit:
```

The **signal capture sample count** $N = \text{samp_rate} \times T = 1.25 \text{ MS/s} \times 8 \text{ s} = 10 \text{ MegaSamples}$ where the signal capture duration T was specified as 8 seconds.

Receiver flowgraph



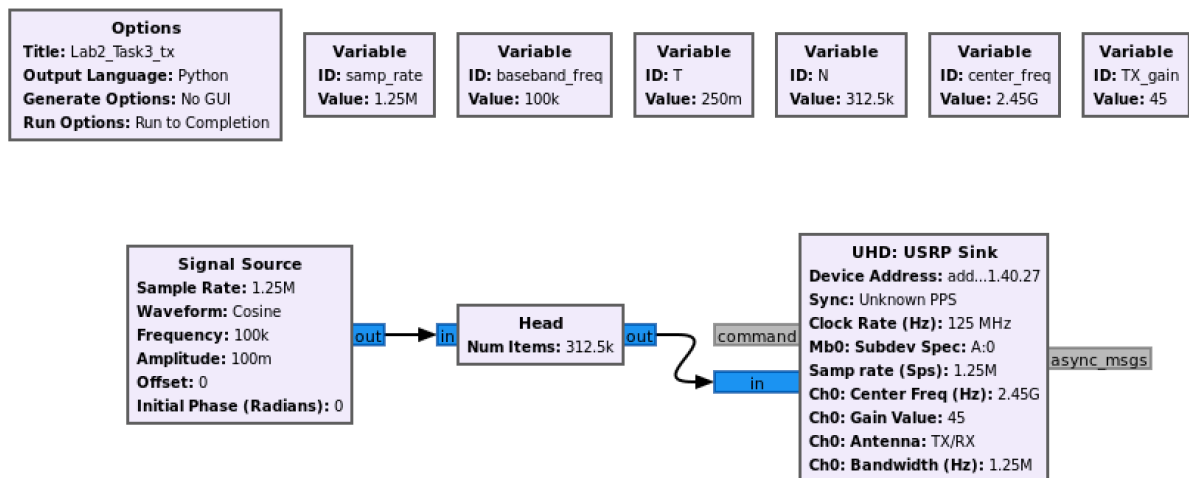
```
[INFO] [UHD] linux; GNU C++ version 13.3.0; Boost_108600; UHD_4.8.0.HEAD-release
[INFO] [MPMD] Initializing 1 device(s) in parallel with args: mgmt_addr=10.61.40.110,type=n3xx,product=n310,serial=3176DD6,name=ni-n3xx-3176DD6,fpga=HG,claimed=False,addr=10.61.40.110
[INFO] [MPM.PeriphManager] init() called with device args `fpga=HG,mgmt_addr=10.61.40.110,name=ni-n3xx-3176DD6,product=n310,clock_source=internal,time_source=internal'.
[INFO] [MULTI_USRP] 1) catch time transition at pps edge
[INFO] [MULTI_USRP] 2) set times next pps (synchronously)
[INFO] [0/Radio#0] Re-initializing dboard to apply bandwidth settings. This may take some time.
[INFO] [0/Radio#0] Bandwidth settings applied: 1250000.000000
Press Enter to quit: █
```

The complex64 file should contain $8N$ bytes = $8 * 10 * 10^6 = 80,000,000$ bytes where N is the signal capture sample count. This can be verified by an `ls -l` of the file.

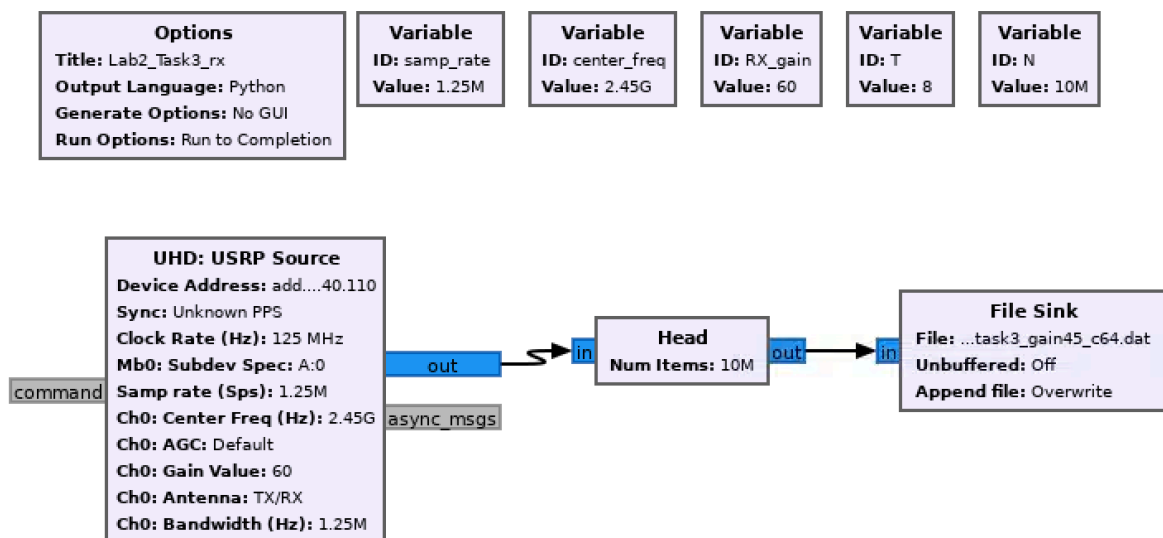
```
In [7]: ls -l work | grep task3_gain40*
-rw-r--r-- 1 marcpham staff 80000000 Sep 12 10:07 task3_gain40_c64.dat
```

Question 4

Transmitter flowgraph: Updated TX_gain to 45 dB



Receiver flowgraph: Updated file sink to go to work/task3_gain45_c64.dat

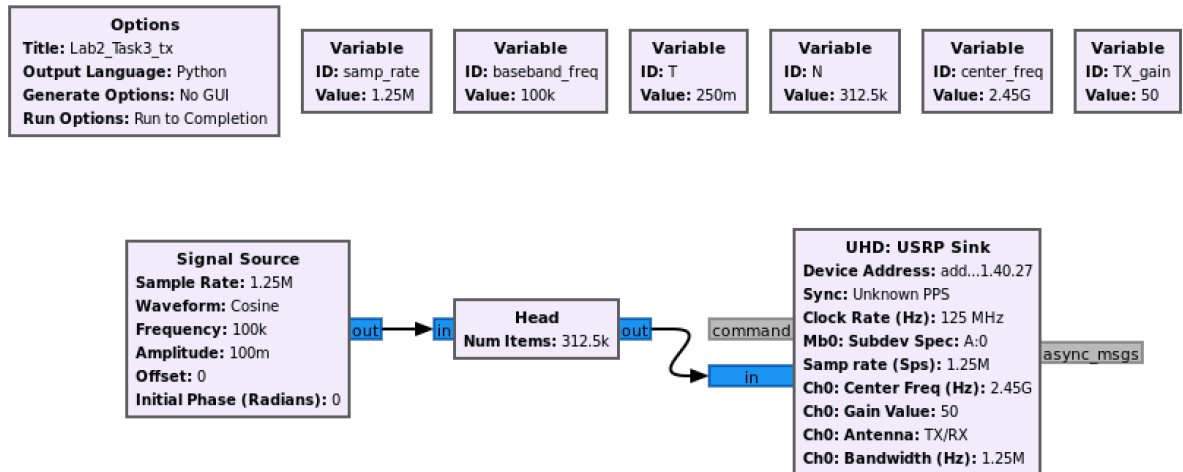


When changing to 45 dB, the complex64 file should still contain 8N bytes = $8 * 10 * 10^6 = 80,000,000$ bytes because the TX_gain doesn't change the number of samples. This can be verified by an ls -l of the file.

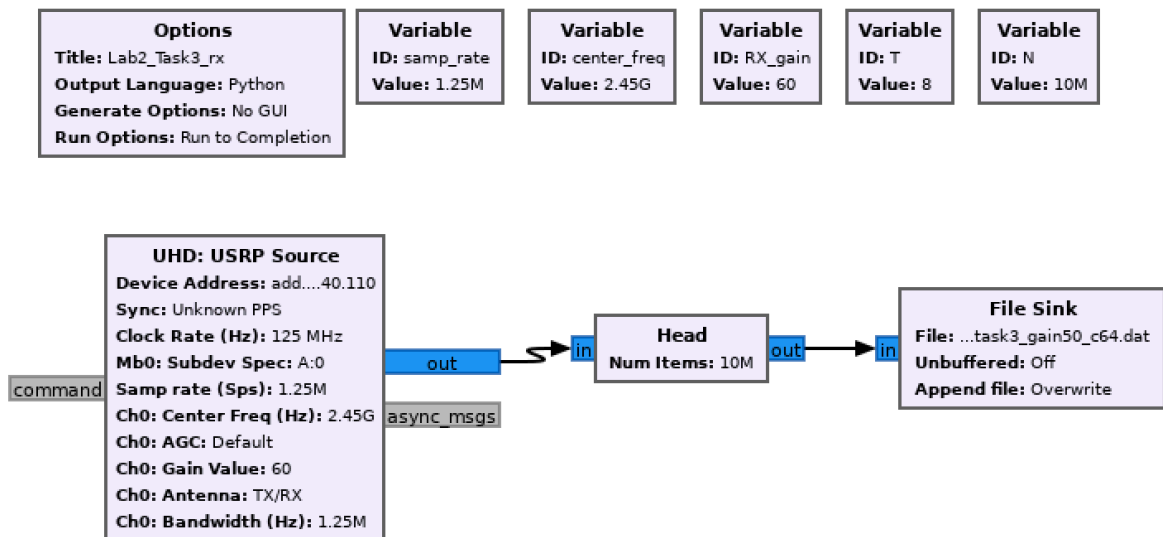
```
In [31]: ls -l work | grep task3_gain45_c64.dat
```

```
-rw-r--r-- 1 marcpam staff 80000000 Sep 12 10:29 task3_gain45_c64.dat
```

Transmitter flowgraph: Updated TX_gain to 50 dB



Receiver flowgraph: Updated file sink to work/task3_gain50_c64.dat



When changing to 50 dB, the complex64 file should still contain 8N bytes = $8 * 10 * 10^6 = 80,000,000$ bytes because the TX_gain doesn't change the number of samples. This can be verified by an ls -l of the file.

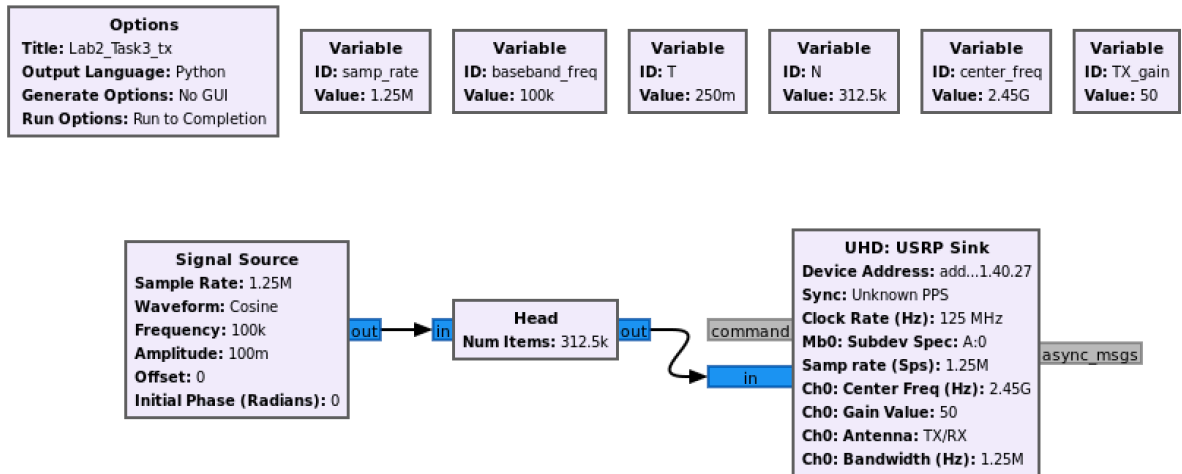
```
In [35]: ls -l work | grep task3_gain50_c64.dat
```

```
-rw-r--r-- 1 marcpam staff 80000000 Sep 12 10:37 task3_gain50_c64.dat
```

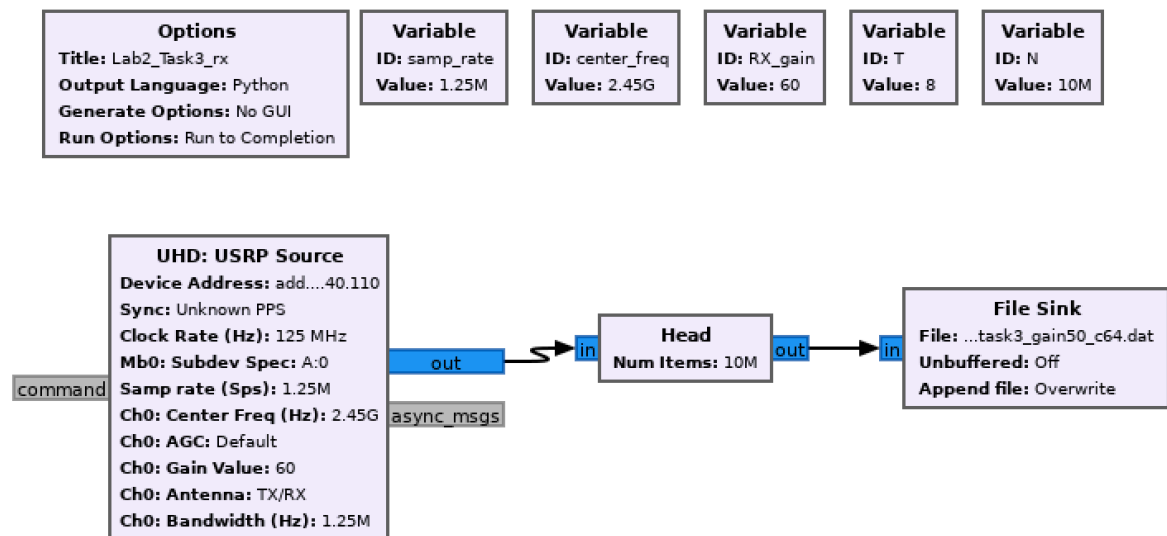
Question 5

Completed flowgraphs

Transmitter flowgraph:



Receiver flowgraph:



5.2: Complex I/Q statistics, power, SNR, and PSD

```
In [167... Fs = 1.25e6
noise = np.fromfile("work/task3_noise_c64.dat", dtype=np.complex64)
gain40 = np.fromfile("work/task3_gain40_c64.dat", dtype=np.complex64)
gain45 = np.fromfile("work/task3_gain45_c64.dat", dtype=np.complex64)
gain50 = np.fromfile("work/task3_gain50_c64.dat", dtype=np.complex64)
```

Burst Detection

After visually inspecting the burst, there was a longer startup transient than shutdown transient, so a longer guard interval was used at the beginning than at the end.

```
In [209... def detect_burst(x, noise_power, block=5000, threshold_db=6):

    # Compare the short-block powers with the transmitter-off noise floor.
    power = np.abs(x)**2
    block_power = np.array([
        np.mean(power[i:i+block])
        for i in range(0, len(x)-block, block)
    ])
    ])
```

```

    thresh = noise_power * 10*(threshold_db/10)

    idx = np.where(block_power > thresh)[0]

    start = idx[0]*block
    end = (idx[-1]+1)*block

    return start, end

def get_burst(x, Fs, noise):

    # Remove guardx ms from the front of the burst and guardy ms from the back of the burst.
    # guardx and guardy > 10 ms
    guardx = int(0.10 * Fs)
    guardy = int(0.015 * Fs)

    Pn = np.var(noise)
    start, end = detect_burst(x, Pn)
    print(f"Start: {(start+guardx)/Fs} seconds")
    print(f"End: {(end-guardy)/Fs} seconds")
    return x[start+guardx:end-guardy], start+guardx, end-guardy

```

```

In [210]: print("TX Gain = 40 Burst")
burst40, start40, end40 = get_burst(gain40, Fs, noise)

print("\nTX Gain = 45 Burst")
burst45, start45, end45 = get_burst(gain45, Fs, noise)

print("\nTX Gain = 50 Burst")
burst50, start50, end50 = get_burst(gain50, Fs, noise)

```

TX Gain = 40 Burst
Start: 5.276 seconds
End: 5.417 seconds

TX Gain = 45 Burst
Start: 5.196 seconds
End: 5.333 seconds

TX Gain = 50 Burst
Start: 6.156 seconds
End: 6.301 seconds

Signal Statistics

```

In [211]: def stats(samples, noise_power):

    mu = np.mean(samples)
    var = np.mean(np.abs(samples-mu)**2)
    Ps = max(var - noise_power, 1e-20)

    snr_db = 10*np.log10(Ps/noise_power)
    normalized_power_db = 10*np.log10(var)

    return mu, var, normalized_power_db, snr_db

```

```

In [212]: mu_n = np.mean(noise)
var_n = np.mean(np.abs(noise-mu_n)**2)
normalized_power_n_db = 10*np.log10(var_n)

mu40,var40,p40,snr40 = stats(burst40,var_n)
mu45,var45,p45,snr45 = stats(burst45,var_n)
mu50,var50,p50,snr50 = stats(burst50,var_n)

print("\nNoise Capture")
print("_" * 40)
print(f"Complex Mean           : {mu_n.real:.6e} + {mu_n.imag:.6e}j")
print(f"Variance                 : {var_n:.6e}")
print(f"Normalized Power (normalized dB) : {normalized_power_n_db:.2f}")

```

```

print(f"SNR (dB)                                : N/A (noise-only capture)")

print("\n40 dB Capture")
print("-" * 40)
print(f"Complex Mean                            : {mu40.real:.6e} + {mu40.imag:.6e}j")
print(f"Variance                                : {var40:.6e}")
print(f"Normalized Power (normalized dB) : {p40:.2f}")
print(f"SNR (dB)                                : {snr40:.2f}")

print("\n45 dB Capture")
print("-" * 40)
print(f"Complex Mean                            : {mu45.real:.6e} + {mu45.imag:.6e}j")
print(f"Variance                                : {var45:.6e}")
print(f"Normalized Power (normalized dB) : {p45:.2f}")
print(f"SNR (dB)                                : {snr45:.2f}")

print("\n50 dB Capture")
print("-" * 40)
print(f"Complex Mean                            : {mu50.real:.6e} + {mu50.imag:.6e}j")
print(f"Variance                                : {var50:.6e}")
print(f"Normalized Power (normalized dB) : {p50:.2f}")
print(f"SNR (dB)                                : {snr50:.2f}")

print("\nMeasured Power Changes")
print("-" * 40)
print(f"40 dB → 45 dB : {p45 - p40:.2f} dB")
print(f"45 dB → 50 dB : {p50 - p45:.2f} dB")
print(f"40 dB → 50 dB : {p50 - p40:.2f} dB")

```

Noise Capture

```

-----
Complex Mean                : 3.526547e-06 + 3.647890e-06j
Variance                    : 2.897758e-07
Normalized Power (normalized dB) : -65.38
SNR (dB)                    : N/A (noise-only capture)

```

40 dB Capture

```

-----
Complex Mean                : 1.589905e-05 + -3.345366e-06j
Variance                    : 5.980981e-02
Normalized Power (normalized dB) : -12.23
SNR (dB)                    : 53.15

```

45 dB Capture

```

-----
Complex Mean                : 4.362754e-06 + -5.872966e-06j
Variance                    : 1.875360e-01
Normalized Power (normalized dB) : -7.27
SNR (dB)                    : 58.11

```

50 dB Capture

```

-----
Complex Mean                : 1.319496e-05 + 5.000123e-06j
Variance                    : 5.834632e-01
Normalized Power (normalized dB) : -2.34
SNR (dB)                    : 63.04

```

Measured Power Changes

```

-----
40 dB → 45 dB : 4.96 dB
45 dB → 50 dB : 4.93 dB
40 dB → 50 dB : 9.89 dB

```

Signal Power Changes

The measured received signal power increased as the transmitter gain was increased from 40 dB to 45 dB and from 45 dB to 50 dB. In a linear system, a 5 dB increase in transmitter gain should produce approximately a 5 dB increase in received signal power because the transmitted signal amplitude is increased while the receiver gain, propagation

loss, and antenna configuration remain constant. Any small deviation from exactly 5 dB may be caused by measurement noise, hardware tolerances, or estimation error.

The measured normalized power values may differ in absolute magnitude due to differences in the transmitter-receiver pair, antenna characteristics, hardware calibration, propagation environment, and how the burst region was selected for analysis. However, normalized power is computed directly from normalized UHD sample amplitudes and is not an absolute RF power measurement. Despite these differences, the measured power increases remain approximately 5 dB for each 5 dB increase in transmitter gain, which is the expected behavior for a linear system.

```
In [213... tx_gain = [40, 45, 50]
rx_power = [p40, p45, p50]

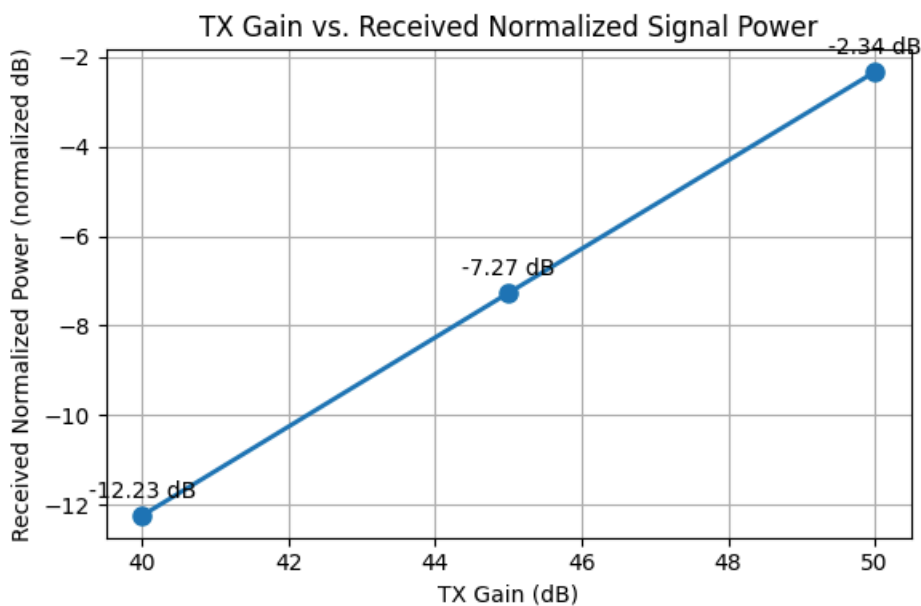
plt.figure(figsize=(6,4))

plt.plot(
    tx_gain,
    rx_power,
    'o-',
    linewidth=2,
    markersize=8
)

for x, y in zip(tx_gain, rx_power):
    plt.annotate(
        f'{y:.2f} dB',
        (x, y),
        textcoords='offset points',
        xytext=(0, 8),
        ha='center'
    )

plt.xlabel('TX Gain (dB)')
plt.ylabel('Received Normalized Power (normalized dB)')
plt.title('TX Gain vs. Received Normalized Signal Power')
plt.grid(True)

plt.tight_layout()
plt.show()
```



Analysis of Clipped Captures

No capture appeared clipped. The maximum observed normalized sample magnitudes were 0.263, 0.459, and 0.841 for the 40 dB, 45 dB, and 50 dB captures, respectively. Since these values remain below the normalized full-scale

value of 1.0, the received signals did not reach the clipping threshold and therefore show no evidence of significant saturation or nonlinear distortion.

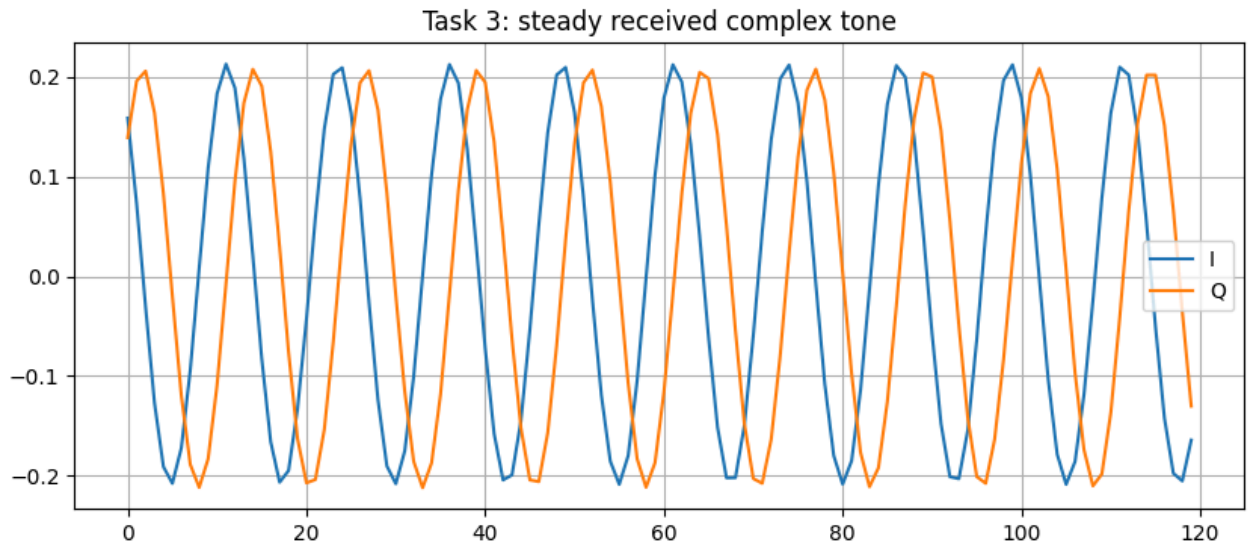
```
In [214... print(np.max(np.abs(burst40)))
print(np.max(np.abs(burst45)))
print(np.max(np.abs(burst50)))
```

```
0.26222715
0.45849672
0.84129643
```

I and Q Plots, Decimated Complex-Plane View & why measured I and Q are not ideal noiseless curves

```
In [215... N = 120

plt.figure(figsize=(10,4))
plt.plot(np.real(burst40[:N]), label='I')
plt.plot(np.imag(burst40[:N]), label='Q')
plt.legend()
plt.grid()
plt.title("Task 3: steady received complex tone")
plt.show()
```



```
In [216... fig, axs = plt.subplots(1, 3, figsize=(15, 5))

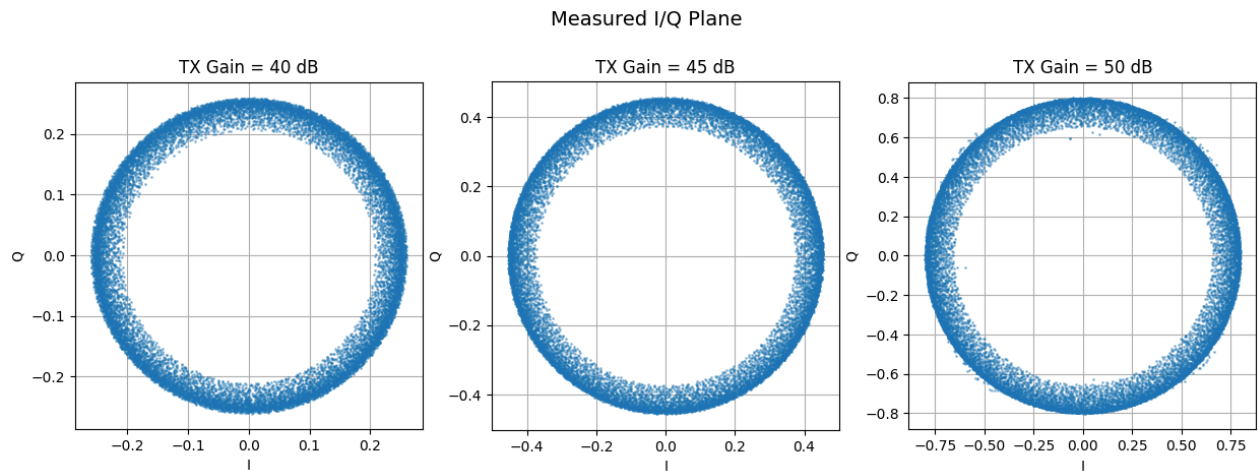
captures = [
    (burst40, "TX Gain = 40 dB"),
    (burst45, "TX Gain = 45 dB"),
    (burst50, "TX Gain = 50 dB"),
]

for ax, (burst, title) in zip(axs, captures):

    ax.plot(
        np.real(burst[:10]),
        np.imag(burst[:10]),
        'r.',
        markersize=2,
        alpha=0.4
    )

    ax.set_title(title)
    ax.set_xlabel('I')
    ax.set_ylabel('Q')
    ax.grid(True)
```

```
ax.set_aspect('equal', adjustable='box')
fig.suptitle(
    'Measured I/Q Plane',
    fontsize=14
)
plt.show()
```



Why are measured I and Q not ideal noiseless curves?

The measured I and Q signals are not ideal noiseless sinusoids because the received signal contains additive receiver noise, hardware imperfections, and propagation effects. Small amplitude fluctuations, phase errors, and DC offsets are introduced by the transmitter and receiver hardware. Consequently, the I-Q plane appears as a noisy cloud rather than a perfect circle, and the I and Q waveforms exhibit deviations from ideal sinusoidal behavior.

How does a noninteger quarter-period sample offset affect a plot of measured I and Q?

The transmitted tone frequency was 100 kHz and the sample rate was 1.25 MS/s. The number of samples in one period is

$$N_{period} = \frac{f_s}{f} = \frac{1.25 \times 10^6}{100 \times 10^3} = 12.5 \text{ samples.}$$

Therefore, one-quarter period corresponds to

$$N_{quarter} = \frac{12.5}{4} = 3.125 \text{ samples.}$$

Since 3.125 samples is not an integer, a quarter-period shift cannot be represented exactly by a whole number of samples. Ideally, the I and Q signals are separated by 90°, which corresponds to a quarter-period offset. However, because the sampling instants occur only at integer sample locations, the receiver cannot place the Q samples exactly one-quarter period away from the I samples.

As a result, the sampled I and Q waveforms will not appear perfectly aligned as ideal cosine and sine waves when plotted. Instead, the measured phase relationship will be only an approximation of 90°, causing small differences in amplitude and phase between the curves. Receiver noise, finite sampling, and hardware imperfections further contribute to deviations from the ideal sinusoidal relationship.

Full and One-Second Reference Captures

The measured bursts exhibit a more gradual rise to their steady-state levels than the reference plots. This behavior is likely caused by a combination of transmitter and receiver startup transients, burst detection beginning before the signal has fully settled, and the use of 10 ms block-power averaging, which smooths rapid changes in signal power. Although the burst edges are less abrupt than those shown in the reference example, a clear steady-state region is present and was isolated using guard intervals before calculating the reported statistics, power, SNR, and PSD measurements.

```
In [222... fig, axs = plt.subplots(3, 1, figsize=(12,10), sharex=False)

captures = [
    (gain40, start40, end40, 'TX Gain = 40 dB'),
    (gain45, start45, end45, 'TX Gain = 45 dB'),
    (gain50, start50, end50, 'TX Gain = 50 dB')
]

window = int(0.010 * Fs)

for ax, (gain, s_start, s_end, title) in zip(axs, captures):
    power_db = []

    for i in range(0, len(gain)-window, window):
        block = gain[i:i+window]

        p = np.mean(np.abs(block)**2)

        power_db.append(
            10*np.log10(p + 1e-20)
        )

    power_db = np.array(power_db)

    t = np.arange(len(power_db))*0.010

    burst_center = (s_start + s_end) / (2 * Fs)

    ax.plot(
        t,
        power_db,
        color='steelblue',
        linewidth=1.5
    )

    ax.axvspan(
        s_start/Fs,
        s_end/Fs,
        color='orange',
        alpha=0.30,
        label='detected burst'
    )

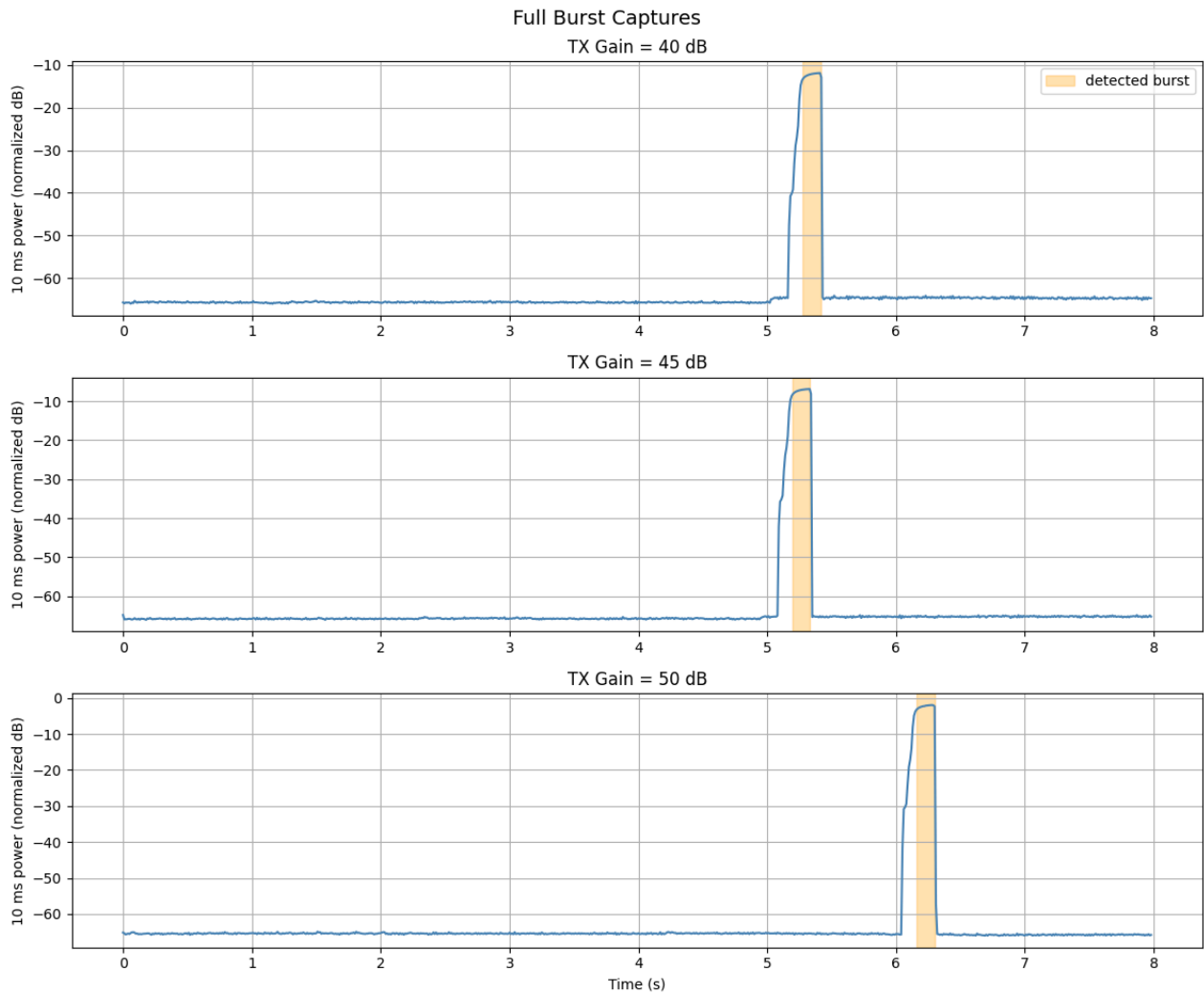
    ax.set_title(title)
    ax.set_ylabel('10 ms power (normalized dB)')
    ax.grid(True)

axs[-1].set_xlabel('Time (s)')

fig.suptitle(
    'Full Burst Captures',
    fontsize=14
)

axs[0].legend()

plt.tight_layout()
plt.show()
```



```
In [221]: fig, axs = plt.subplots(3, 1, figsize=(12,10), sharex=False)

captures = [
    (gain40, start40, end40, 'TX Gain = 40 dB'),
    (gain45, start45, end45, 'TX Gain = 45 dB'),
    (gain50, start50, end50, 'TX Gain = 50 dB')
]

window = int(0.010 * Fs)

for ax, (gain, s_start, s_end, title) in zip(axs, captures):

    power_db = []

    for i in range(0, len(gain)-window, window):

        block = gain[i:i+window]

        p = np.mean(np.abs(block)**2)

        power_db.append(
            10*np.log10(p + 1e-20)
        )

    power_db = np.array(power_db)

    t = np.arange(len(power_db))*0.010

    burst_center = (s_start + s_end) / (2 * Fs)
```

```

xmin = np.floor(burst_center)
xmax = xmin + 1

ax.plot(
    t,
    power_db,
    color='steelblue',
    linewidth=1.5
)

ax.axvspan(
    s_start/Fs,
    s_end/Fs,
    color='orange',
    alpha=0.30,
    label='detected burst'
)

ax.set_xlim(xmin, xmax)

ax.set_title(title)
ax.set_ylabel('10 ms power (normalized dB)')
ax.grid(True)

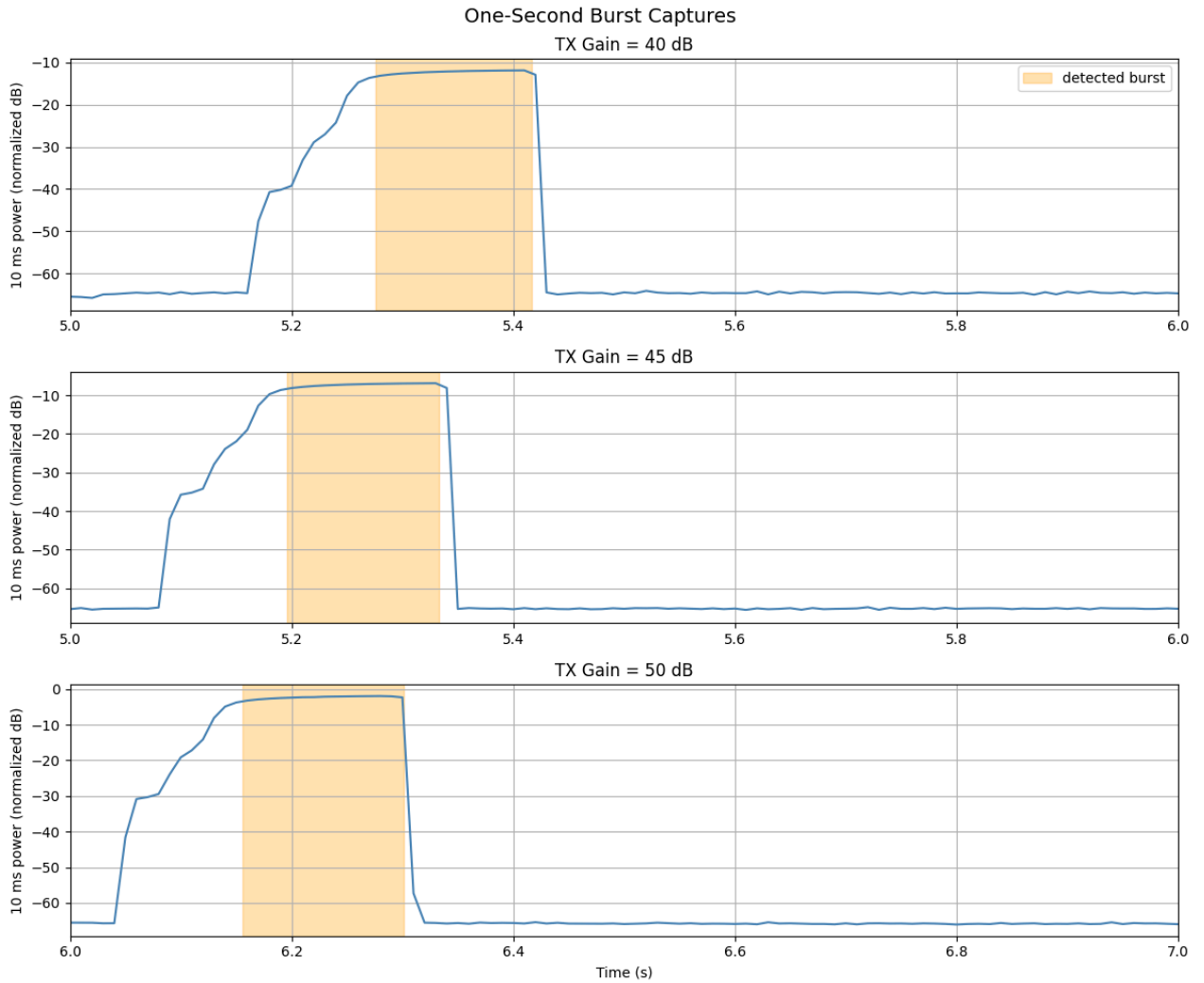
axs[-1].set_xlabel('Time (s)')

fig.suptitle(
    'One-Second Burst Captures',
    fontsize=14
)

axs[0].legend()

plt.tight_layout()
plt.show()

```



Noise / Burst PSDs & Measured Baseband Peaks

The noise-only PSD is significantly lower than the burst PSD because the received signal power is much larger than the receiver noise power. Although the transmitted waveform is a single tone centered near 100 kHz, the finite duration of the burst and the finite-length Welch PSD estimate spread some signal energy into neighboring frequency bins. As a result, the burst PSD exhibits both a strong peak at approximately 100 kHz and an elevated spectral floor compared to the noise-only capture. The higher absolute received power levels observed in this experiment further increase this effect relative to the reference results.

The measured baseband peak frequency was identified from the maximum of the Welch PSD for each capture. The corresponding RF frequency was calculated by adding the measured baseband frequency to the center frequency of 2.45 GHz:

$$f_{RF} = f_c + f_{BB}$$

where

$$f_c = 2.45 \text{ GHz}$$

TX Gain = 40 dB

Measured baseband peak frequency:

$$f_{BB} = 99.64 \text{ kHz}$$

Corresponding RF frequency:

$$f_{RF} = 2.45 \text{ GHz} + 99.64 \text{ kHz} = 2.45009964 \text{ GHz}$$

Peak PSD:

$$-37.16 \text{ dB/Hz}$$

TX Gain = 45 dB

Measured baseband peak frequency:

$$f_{BB} = 99.79 \text{ kHz}$$

Corresponding RF frequency:

$$f_{RF} = 2.45 \text{ GHz} + 99.79 \text{ kHz} = 2.45009979 \text{ GHz}$$

Peak PSD:

$$-32.49 \text{ dB/Hz}$$

TX Gain = 50 dB

Measured baseband peak frequency:

$$f_{BB} = 99.64 \text{ kHz}$$

Corresponding RF frequency:

$$f_{RF} = 2.45 \text{ GHz} + 99.64 \text{ kHz} = 2.45009964 \text{ GHz}$$

Peak PSD:

$$-27.32 \text{ dB/Hz}$$

A small **frequency offset** was observed because the measured baseband peak frequencies (99.64 kHz and 99.79 kHz) were slightly different from the expected 100 kHz tone frequency. This offset is common in practical systems and is caused by small frequency inaccuracies in the transmitter and receiver oscillators, as well as estimation error in the PSD measurement process.

The PSD plots show a strong spike near 0 Hz, which is evidence of **DC leakage**. DC leakage occurs when a small DC component is introduced by the transmitter or receiver hardware. In direct-conversion SDR systems, imperfections such as local oscillator leakage and analog circuit offsets can produce energy at 0 Hz even when no signal is intentionally transmitted there.

The measured PSD was computed using Welch's method with a finite FFT length, so it has **finite PSD resolution**. Because the frequency bins are spaced by a finite amount, the measured peak does not necessarily occur exactly at 100 kHz. The finite observation time also causes spectral leakage, which spreads signal energy into nearby frequency bins and broadens the appearance of the spectral peak.

In [220...

```
# Noise PSD
f_noise, S_noise = signal.welch(
    noise,
    fs=Fs,
    window='hann',
    nperseg=8192,
    noverlap=4096,
    return_onesided=False,
    scaling='density'
)

f_noise = np.fft.fftshift(f_noise)
S_noise = np.fft.fftshift(S_noise)
noise_psd_db = 10*np.log10(S_noise + 1e-20)
```

```

fig, axs = plt.subplots(3, 1, figsize=(12, 10), sharex=True)

captures = [
    (burst40, "TX Gain = 40 dB"),
    (burst45, "TX Gain = 45 dB"),
    (burst50, "TX Gain = 50 dB"),
]

for ax, (burst, title) in zip(axs, captures):

    f, Srr = signal.welch(
        burst,
        fs=Fs,
        window='hann',
        nperseg=8192,
        noverlap=4096,
        return_onesided=False,
        scaling='density'
    )

    f = np.fft.fftshift(f)
    Srr = np.fft.fftshift(Srr)

    psd_db = 10*np.log10(Srr + 1e-20)

    # Burst PSD
    ax.plot(
        f/1e3,
        psd_db,
        linewidth=1.5,
        label='Burst PSD'
    )

    # Noise PSD
    ax.plot(
        f_noise/1e3,
        noise_psd_db,
        '--',
        linewidth=1.2,
        color='black',
        alpha=0.8,
        label='Noise PSD'
    )

    peak_idx = np.argmax(Srr)
    peak_freq = f[peak_idx] / 1e3
    peak_db = psd_db[peak_idx]

    ax.plot(
        peak_freq,
        peak_db,
        'ro',
        markersize=4
    )

    ax.axvline(
        peak_freq,
        color='red',
        linestyle='--',
        alpha=0.8
    )

    print(title)
    print(f'Baseband Peak Frequency: {peak_freq:.2f} kHz')
    print(f'Peak PSD: {peak_db:.2f} dB/Hz\n')

    ax.text(
        0.98,
        0.95,

```

```

        f'Peak Frequency: {peak_freq:.2f} kHz\n'
        f'Peak PSD: {peak_db:.2f} dB/Hz',
        transform=ax.transAxes,
        ha='right',
        va='top',
        fontsize=8,
        bbox=dict(
            facecolor='white',
            edgecolor='red',
            boxstyle='round,pad=0.3',
            alpha=0.9
        )
    )

    ax.set_title(title)
    ax.set_ylabel('PSD (dB/Hz)')
    ax.set_xlim(-200, 200)
    ax.grid(True)

    axs[0].legend(loc='upper left')

    axs[-1].set_xlabel('Frequency (kHz)')

    fig.suptitle(
        'Welch PSD of Noise and Steady-State Bursts',
        fontsize=14
    )

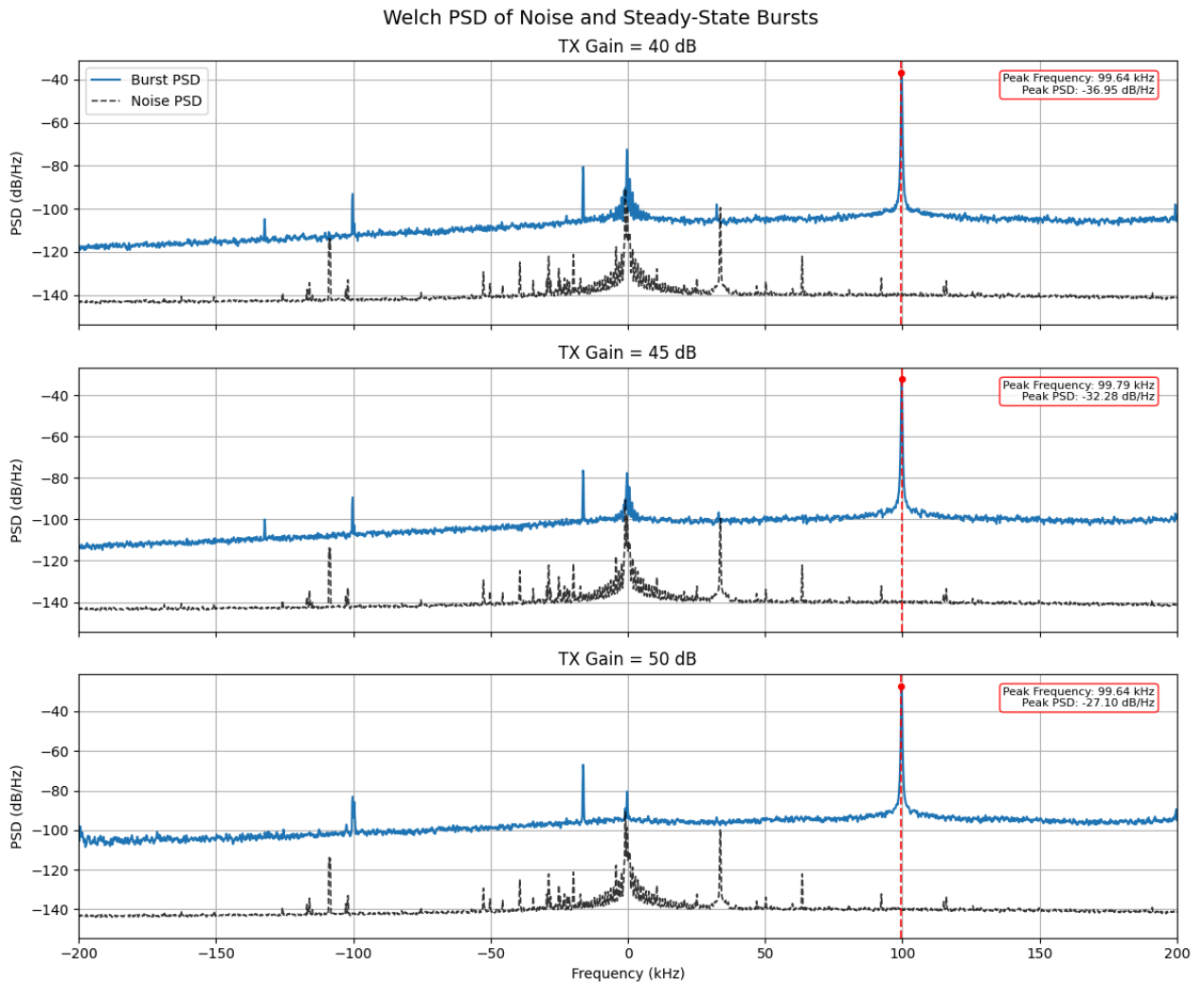
    plt.tight_layout()
    plt.show()

```

TX Gain = 40 dB
 Baseband Peak Frequency: 99.64 kHz
 Peak PSD: -36.95 dB/Hz

TX Gain = 45 dB
 Baseband Peak Frequency: 99.79 kHz
 Peak PSD: -32.28 dB/Hz

TX Gain = 50 dB
 Baseband Peak Frequency: 99.64 kHz
 Peak PSD: -27.10 dB/Hz



5. Task 4: Sampling-Rate Effects with Measured Data

```
In [223... x = burst45

raw_ds2 = x[:,2]
raw_ds10 = x[:,10]
filtered_ds10 = signal.resample_poly(x, up=1, down=10)
```

The filtered factor-10 PSD reports a peak frequency of -16.28 kHz because a maximum PSD value must exist even when only noise is present. Since the anti-aliasing filter removes the original $+100$ kHz tone before decimation, the remaining spectrum is dominated by noise. Therefore, the reported peak does not indicate the presence of a true signal and instead corresponds to a random noise fluctuation.

```
In [227... signals = [
    (x, 1.25e6, "Original", 625),
    (raw_ds2, 625e3, "Raw downsample by 2", 312.5),
    (raw_ds10, 125e3, "Raw downsample by 10", 62.5),
    (filtered_ds10, 125e3, "Filtered downsample by 10", 62.5),
]

fig, axs = plt.subplots(2, 2, figsize=(12, 9))
axs = axs.ravel()

for ax, (sig, fs, title, nyquist) in zip(axs, signals):

    f, Srr = signal.welch(
        sig,
```

```

        fs=fs,
        window='hann',
        nperseg=min(8192, len(sig)),
        noverlap=min(4096, len(sig)//2),
        return_onesided=False,
        scaling='density'
    )

    f = np.fft.fftshift(f)
    Srr = np.fft.fftshift(Srr)

    psd_db = 10*np.log10(Srr + 1e-20)

    peak_idx = np.argmax(Srr)
    peak_freq = f[peak_idx] / 1e3

    ax.plot(
        f/1e3,
        psd_db,
        linewidth=1.3
    )

    ax.axvline(
        peak_freq,
        color='red',
        linestyle='--',
        alpha=0.7
    )

    ax.set_title(title)
    ax.set_xlabel('Frequency (kHz)')
    ax.set_ylabel('Normalized PSD (dB/Hz)')

    ax.set_xlim(-nyquist, nyquist)

    ax.text(
        0.98,
        0.95,
        f'Peak: {peak_freq:.2f} kHz',
        transform=ax.transAxes,
        ha='right',
        va='top',
        fontsize=8,
        bbox=dict(
            facecolor='white',
            alpha=0.8
        )
    )

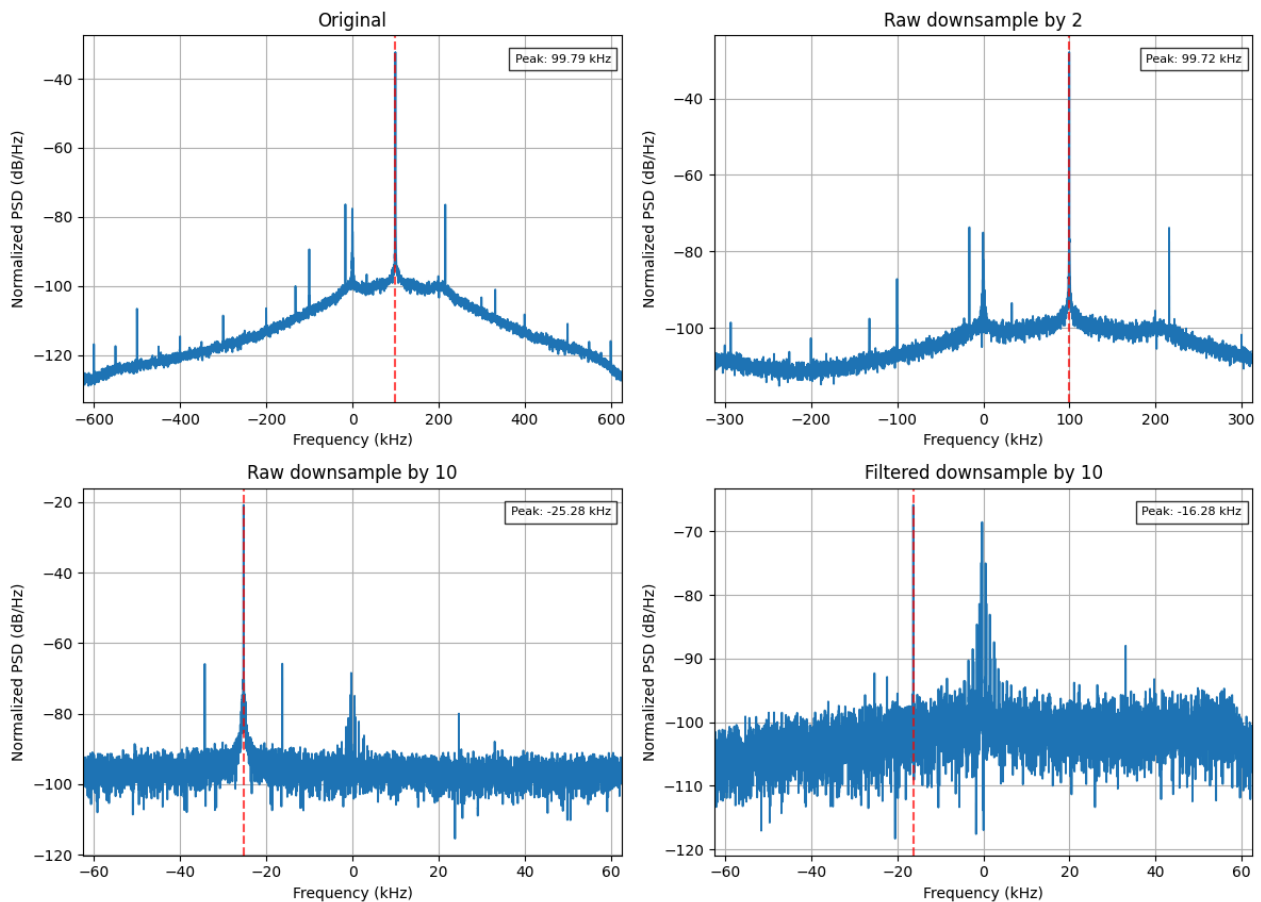
    ax.grid(True)

plt.suptitle(
    'Sampling-Rate Effects on Measured Data',
    fontsize=14
)

plt.tight_layout()
plt.show()

```

Sampling-Rate Effects on Measured Data



The I-waveform plots were normalized by the peak amplitude rather than the RMS value. For a pure sinusoid, RMS normalization would produce peak values of

$$\pm\sqrt{2} \approx \pm 1.414,$$

while peak normalization produces values between approximately

$$-1 \text{ and } +1.$$

Peak normalization was chosen because it more closely matches the scale used in the reference figure (despite what the y-axes say) and allows the waveform shape and sampling-rate effects to be compared more directly. The choice of normalization does not affect the observed aliasing behavior or the relative differences between the sampling-rate cases; it only changes the vertical scaling of the plots.

```
In [233... signals = [
    (x, 1.25e6, "Original"),
    (raw_ds2, 625e3, "Raw downsample by 2"),
    (raw_ds10, 125e3, "Raw downsample by 10"),
    (filtered_ds10, 125e3, "Filtered downsample by 10"),
]

fig, axs = plt.subplots(4, 1, figsize=(12, 12))

duration_us = 80
duration = duration_us * 1e-6

for ax, (sig, fs, title) in zip(axs, signals):

    # Plot the same physical duration (80 us) for every signal
    N = min(int(duration * fs), len(sig))

    # Extract I component
    I = np.real(sig[:N])
```

```

# Peak Normalization
amp = np.max(np.abs(I))
I_norm = I / amp

# RMS Normalization
# rms = np.sqrt(np.mean(I**2))
# I_norm = I / rms

# Time axis
t_us = np.arange(N) / fs * 1e6

ax.plot(
    t_us,
    I_norm,
    '.-',
    markersize=4,
    linewidth=1.2
)

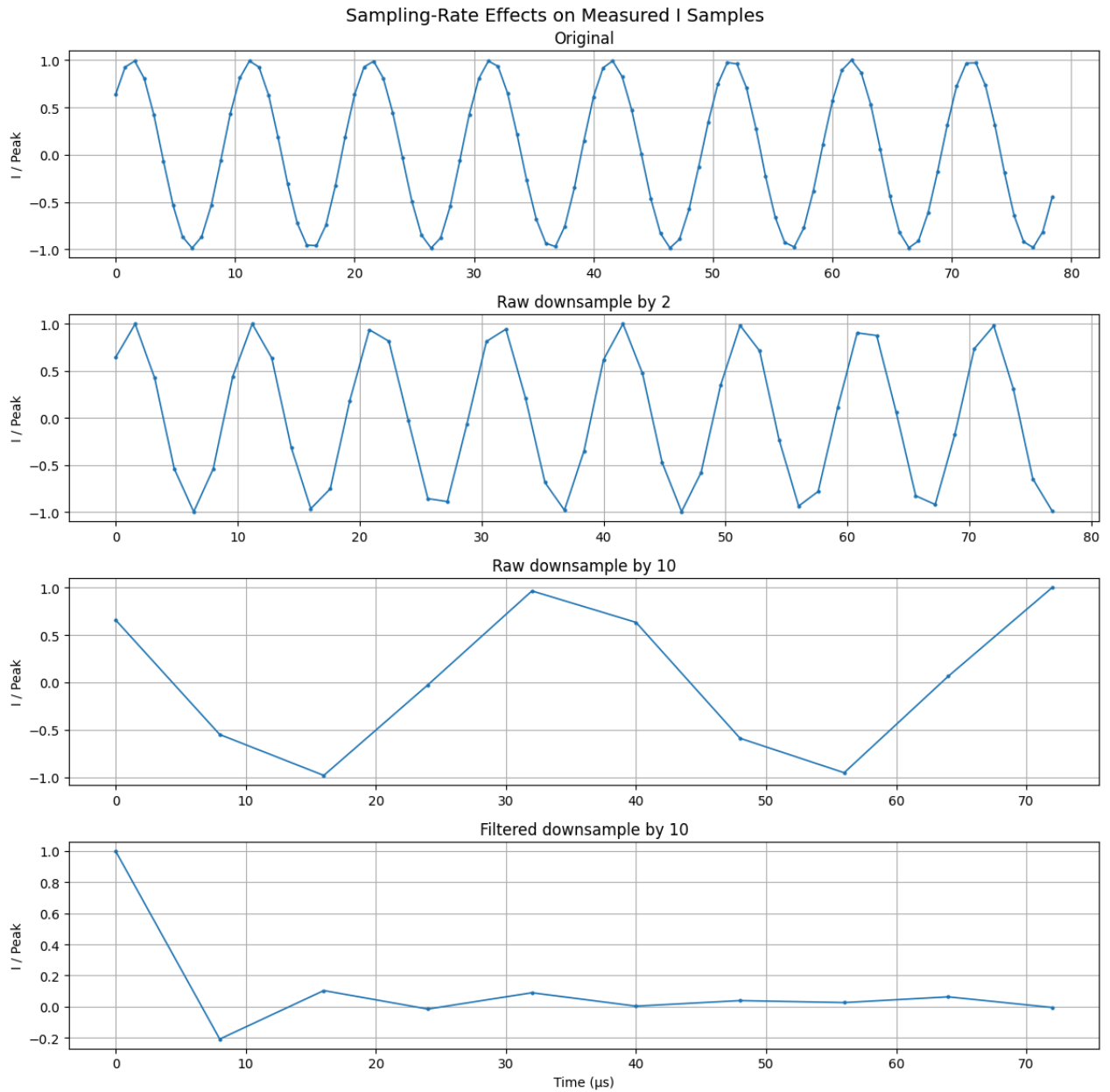
ax.set_title(title)
ax.set_ylabel('I / Peak')
ax.grid(True)

axs[-1].set_xlabel('Time (μs)')

fig.suptitle(
    'Sampling-Rate Effects on Measured I Samples',
    fontsize=14
)

plt.tight_layout()
plt.show()

```



Peak Comparisons

The alias frequency is given by $f_{\text{alias}} = \left[\left(f + \frac{f_s}{2} \right) \bmod f_s \right] - \frac{f_s}{2}$

The original tone frequency is $f = 100\text{kHz}$, and the original sample rate was $f_s = 1.25\text{ MS/s}$

Raw Downsample by 2

After downsampling by 2, the new sample rate is $f_s = \frac{1.25\text{ MS/s}}{2} = 625\text{ kS/s}$.

Applying the alias equation, $f_{\text{alias}} = [(100 + 312.5) \bmod 625] - 312.5 = 412.5 - 312.5 = 100\text{ kHz}$

Because 100 kHz already lies within the new Nyquist interval $[-312.5, 312.5)\text{ kHz}$, no aliasing occurs.

Prediction: $f_{\text{alias}} = +100\text{ kHz}$

Raw Downsample by 10

After downsampling by 10, the new sample rate is $f_s = \frac{1.25\text{ MS/s}}{10} = 125\text{ kS/s}$.

Applying the alias equation, $f_{\text{alias}} = [(100 + 62.5) \bmod 125] - 62.5 = 37.5 - 62.5 = -25 \text{ kHz}$

Since 100 kHz lies outside the new Nyquist interval $[-62.5, 62.5) \text{ kHz}$, the tone aliases to -25 kHz .

Prediction: $f_{\text{alias}} = -25 \text{ kHz}$

The **measured PSD peak frequencies** are calculated below. The original signal and the factor-2 downsample both produced peaks near +100 kHz, while the factor-10 raw downsample produced a peak near -25 kHz as predicted by the alias equation. The measured PSD peak frequencies were very close to the predicted alias frequencies, with only small differences observed between the calculated and measured values. These discrepancies are expected because the PSD was estimated using Welch's method with a finite FFT length, resulting in finite frequency resolution.

The filtered factor-10 PSD reports a peak frequency of -16.28 kHz because a maximum PSD value must exist even when only noise is present. Since the anti-aliasing filter removes the original +100 kHz tone before decimation, the remaining spectrum is dominated by noise. Therefore, the reported peak does not indicate the presence of a true signal and instead corresponds to a random noise fluctuation.

The negative sign indicates that the aliased tone appears at a negative frequency in the complex baseband representation. After downsampling by 10, the new Nyquist interval becomes -62.5 kHz to $+62.5 \text{ kHz}$. The original +100 kHz tone lies outside this interval, so it must fold back into the valid frequency range. As a result, the +100 kHz tone aliases to -25 kHz rather than remaining at +100 kHz. For a complex signal, positive and negative frequencies are distinct and represent different directions of rotation in the complex plane. Therefore, the aliased tone retains its negative frequency and appears at -25 kHz in the PSD. If the signal were real-valued, positive and negative frequencies would not be distinguishable, making the sign information unavailable.

```
In [234... def get_psd_peak(x, Fs):  
  
    f, Srr = signal.welch(  
        x,  
        fs=Fs,  
        window='hann',  
        nperseg=8192,  
        noverlap=4096,  
        return_onesided=False,  
        scaling='density'  
    )  
  
    f = np.fft.fftshift(f)  
    Srr = np.fft.fftshift(Srr)  
  
    peak_idx = np.argmax(Srr)  
  
    return f, Srr, f[peak_idx]
```

```
In [235... _, _, peak_orig = get_psd_peak(x, 1.25e6)  
_, _, peak_ds2 = get_psd_peak(raw_ds2, 625e3)  
_, _, peak_ds10 = get_psd_peak(raw_ds10, 125e3)  
_, _, peak_filt10 = get_psd_peak(filtered_ds10, 125e3)  
  
print("\nMeasured PSD Peak Frequencies")  
print("=" * 45)  
  
print(f"Original Signal")  
print(f"  Peak Frequency : {peak_orig/1e3:.2f} kHz")  
  
print(f"\nRaw Downsample by 2")  
print(f"  Peak Frequency : {peak_ds2/1e3:.2f} kHz")  
  
print(f"\nRaw Downsample by 10")  
print(f"  Peak Frequency : {peak_ds10/1e3:.2f} kHz")
```

```
print(f"\nFiltered Downsample by 10")
print(f" Peak Frequency : {peak_filt10/1e3:.2f} kHz")
```

Measured PSD Peak Frequencies

=====

Original Signal

Peak Frequency : 99.79 kHz

Raw Downsample by 2

Peak Frequency : 99.72 kHz

Raw Downsample by 10

Peak Frequency : -25.28 kHz

Filtered Downsample by 10

Peak Frequency : -16.28 kHz

Raw vs. Filtered Factor-10 Output Powers

In [240...

```
mu = np.mean(raw_ds10)
P_raw10 = np.mean(np.abs(raw_ds10 - mu)**2)

mu = np.mean(filtered_ds10)
P_filt10 = np.mean(np.abs(filtered_ds10 - mu)**2)

power_raw10_db = 10*np.log10(P_raw10)
power_filt10_db = 10*np.log10(P_filt10)

print(f"Raw factor-10 power      : {power_raw10_db:.2f} dB")
print(f"Filtered factor-10 power : {power_filt10_db:.2f} dB")
print(f"Difference                : {power_raw10_db - power_filt10_db:.2f} dB")
```

Raw factor-10 power : -7.27 dB

Filtered factor-10 power : -46.42 dB

Difference : 39.15 dB

The measured normalized power of the raw factor-10 signal was $P_{\text{raw}} = -7.27$ dB, while the measured normalized power of the filtered factor-10 signal was $P_{\text{filtered}} = -46.42$ dB. The difference between the two measurements was $\Delta P = 39.15$ dB.

The raw factor-10 downsample retained significant signal power because no anti-aliasing filter was applied before decimation. Although the original +100 kHz tone lies outside the new Nyquist interval of -62.5 kHz to +62.5 kHz, it is not removed. Instead, it aliases to -25 kHz and remains visible in the PSD as a strong spectral peak.

In contrast, the filtered factor-10 downsample applies an anti-aliasing filter before selecting every tenth sample. This filter attenuates frequencies above the new Nyquist limit of +62.5 kHz, including the original +100 kHz tone. As a result, the tone is largely removed and the output power decreases substantially, causing the PSD to approach the receiver noise floor.

Could the Original +100 kHz Tone Be Identified?

No. From the raw 125 kS/s samples alone, the observed -25 kHz tone does not uniquely identify the original signal frequency. Multiple frequencies can alias to the same observed frequency after downsampling. For example, +100 kHz aliases to -25 kHz when sampled at 125 kS/s, but so do +225 kHz, +350 kHz, and other frequencies separated by integer multiples of 125 kHz. Similarly, frequencies such as -25 kHz and -150 kHz could also produce the same observed aliased frequency. Without prior knowledge of the original signal bandwidth or the use of an anti-aliasing filter, there is no way to determine which of these possible original frequencies produced the observed -25 kHz tone. Therefore, the raw 125 kS/s samples alone cannot uniquely identify the original +100 kHz source.

Conclusion

This lab demonstrated how noise, sampling, aliasing, filtering, and spectral analysis affect both simulated and measured wireless signals. Statistical tools such as mean, variance, power, and SNR were used to characterize deterministic and noisy signals, showing how random noise increases uncertainty in signal measurements and raises the spectral noise floor. Comparing clean and noisy signals highlighted the importance of estimating signal statistics from finite records rather than relying solely on theoretical values.

The FFT and Welch PSD were used to analyze signals in the frequency domain. The FFT provided a direct view of discrete frequency components, while the PSD provided a more robust estimate of how signal power is distributed across frequency. Measured N310 captures showed expected effects such as frequency offset, DC leakage, finite PSD resolution, and noise. The PSDs also demonstrated how increasing transmitter gain increased the received signal power while preserving the same underlying tone frequency. Together, these results showed how frequency-domain analysis can be used to identify signals, estimate power, and evaluate system performance in the presence of noise.

Sampling and filtering concepts were explored through both synthetic examples and measured SDR data. The aliasing experiments demonstrated that signals outside the Nyquist interval fold into the observable bandwidth after downsampling. Raw downsampling preserved aliased signal energy, while filtered downsampling removed out-of-band content before decimation and prevented aliasing. These results reinforced the importance of anti-alias filtering and showed that sampled data alone may not uniquely reveal the original signal frequency without prior bandwidth information.

The measured N310 recordings provided practical experience with complex I/Q samples, GNU Radio, UHD, and software-defined radio hardware. GNU Radio flowgraphs were used to generate, transmit, receive, and store complex baseband signals, while UHD provided the interface between software and the USRP N310 radios. Analysis of the measured I and Q signals demonstrated how complex sampling preserves both amplitude and phase information, allowing positive and negative frequencies to be distinguished. Overall, the lab connected theoretical signal-processing concepts with real SDR measurements and provided a foundation for future work involving modulation, synchronization, channel estimation, OFDM, and MIMO systems.